

在黑暗中进化智能体：基于自偏好的回顾式技能框架优化

Wenbo Pan^{1*} Shujie Liu² Chin-Yew Lin² Jingying Zeng² Xianfeng Tang²
Xiangyang Zhou² Yan Lu² Xiaohua Jia¹

¹City University of Hong Kong ²Microsoft Research Asia

摘要

人工智能智能体依赖由技能、工具和工作流构成的技能框架来解决复杂问题。持续改进这一技能框架对于适应新任务至关重要。然而，现有优化方法通常需要带有真实标签的验证集，但在实际部署环境中，此类标注数据难以获取。为解决这一问题，本文提出回顾式技能框架优化（Retrospective Harness Optimization, RHO），这是一种仅利用历史轨迹的自监督方法。具体而言，RHO 从历史轨迹中筛选出多样化的困难任务核心集，并并行重新求解这些任务。智能体通过自验证和自洽性分析这些执行记录，随后生成候选技能框架更新，并依据自身的成对自偏好选择最有效的更新。我们在软件工程、技术工作和知识工作三个不同领域对 RHO 进行了评估。值得注意的是，仅一轮优化即可将 SWE-Bench Pro 的通过率从 59% 提升至 78%，且无需任何外部评分。此外，分析表明 RHO 能有效针对先前的失败模式。因此，优化后的技能框架改变了智能体的行为模式，并在长时程会话中保持更高的准确率。代码可在 <https://github.com/wbopan/retro-harness> 获取，项目网站位于 <https://paper-rho.wenbo.io>。

1 引言

技能框架通过为人工智能智能体提供可用技能、工作流和工具，使其能够完成复杂任务。一个重要的研究问题是如何持续改进技能框架。具体而言，在智能体部署后，我们旨在持续进化其技能框架

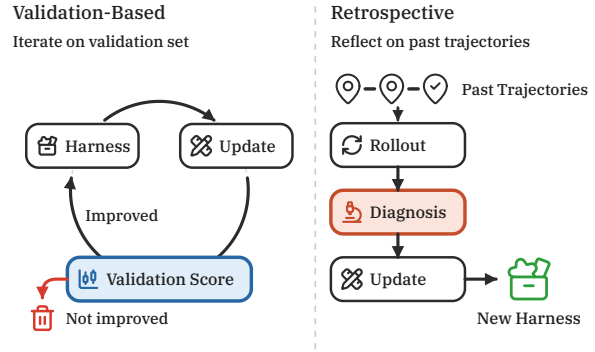


图 1: RHO 与基于验证反馈的技能框架优化对比。前者针对带标签的验证集进行优化，而 RHO 在单次回顾过程中利用历史轨迹进行优化，无需真实标签。

通过从过往经验中学习，进而提升其在未来任务上的性能。先前工作已提出多种演化智能体框架的方法（Zhou et al., 2022; Yang et al., 2023; Khattab et al., 2023; Yuksekgonul et al., 2024; Agrawal et al., 2025; Hu et al., 2024; Lee et al., 2026）。然而，这些方法依赖验证集评分来指导改进。在实际部署场景中，通常难以收集一个能准确估计未来任务分布的验证集来验证更新后的框架。另一方面，智能体的持续运行自然会产生大量来自过往任务的轨迹。这引出了我们的核心问题：在仅能访问过往轨迹的情况下，能否改进智能体框架以提升未来性能？为解决这一问题，本文提出回顾式框架优化（Retrospective Harness Optimization, RHO），一种通过回顾分析过往轨迹来优化框架的自监督方法。该方法利用智能体对轨迹的内部自我偏好来指导优化过程。图 1 对比了 RHO 与传统的验证反馈优化。

*通讯作者。工作完成于实习期间于微软亚洲研究院。

该过程针对带标签的验证集进行迭代。图 2 展示了这一流程。具体而言，给定大量历史轨迹，RHO 首先筛选出一个多样且具有挑战性的任务核心集。随后，智能体对核心集中的每个任务进行多次重试，以生成并行轨迹。在此基础上，我们提取两类诊断信号，即轨迹内的自验证和跨并行轨迹的自洽性。这些信号随后被用于指导生成框架更新。最后，利用智能体的成对自偏好，从新生成的提案中选出最有前景的框架。我们在软件工程、技术工作和知识工作三个智能体领域评估了 RHO 的有效性。RHO 在所有三个领域均持续提升智能体的性能。值得注意的是，仅对软件工程轨迹运行一轮回顾性框架优化，我们就将 SWE-Bench Pro (Deng 等, 2025) 的通过率从 59% 提升至 78%，且无需依赖验证集评分。此外，我们详细分析了回顾性优化过程如何提升性能。我们观察到，RHO 针对过去任务中常见的失败模式设计了特定的技能和工具。这些组件重塑了智能体的行动模式，并帮助其在长时程会话中保持更高的准确率。此外，我们定量分析了回顾性过程中诊断信号的贡献。该分析表明，RHO 中的每一步都逐步隔离出有助于性能提升的信号。我们的贡献如下：◇ 我们提出了回顾性框架优化，解决了仅从未标记轨迹中改进完整框架（包括记忆、上下文、技能和工具）的空白。◇ 我们在三个场景中评估了 RHO，并表明回顾性分析始终优于直接的经验积累，且以约三分之一的算力预算达到了与验证反馈驱动进化相当的质量，且无需标签。◇ 我们定量分析了框架优化对智能体性能的影响，表明收集有效的改进信号可导致框架的针对性变化，并优化智能体的行为。

2 相关工作

框架优化。框架优化通过编辑围绕固定模型的提示、程序参数或工作流代码来改进智能体。一条路线针对带标签的度量优化提示或流水线参数，涵盖大语言模型即优化器搜索 (Yang et al., 2023)、声明式流水线编译 (Khattab et al., 2023)、文本梯度更新 (Yuksekgonul et al., 2024)、反思式提示演化 (Agrawal et al., 2025) 以及跨模型提示演化 (Wu et al., 2026)。另一条更具智能体性的路线让元智能体重写智能体自身的代码，其中 ADAS 搜索智能体系统设计空间 (Hu et al., 2024)、Meta-Harness 利用先前候选的执行轨迹和分数在框架代码上进行搜索 (Lee et al., 2026)，而 M^{*} 将记忆系统演化为可执行程序，在带标签的验证集上对候选记忆程序进行评分 (Pan et al., 2026a)。尽管这些方法在编辑的表面内容上有所不同，但它们都使用带标签的验证度量来引导搜索。RHO 偏离了这一范式，无需验证反馈，并在对未标注的历史轨迹进行单次回顾性处理中改进框架。

智能体自我改进。第二条路线利用智能体自身的过往经验来改进智能体，使用智能体对轨迹的自我判断代替真实标签。动态备忘单在测试时维护一个自我管理的可复用策略和代码片段记忆库 (Suzgun et al., 2025)，而推理银行从自我判断的成功与失败中提炼可泛化的推理策略 (Ouyang et al., 2025)。MemMA 协调多个智能体的记忆循环，并针对自我生成的探针问题修复其记忆库 (Lin et al., 2026)，睡眠时间计算则在查询到达之前离线预计算有用的上下文 (Lin et al., 2025)。与我们的工作同时，SkillOS 则利用结果和评判奖励的强化学习训练技能管理者，从累积经验中更新技能库 (Ouyang et al., 2026)。这些方法丰富了智能体存储的记忆、上下文或技能列表，而框架的其余部分保持不变。相比之下，RHO 优化整个框架，包括可执行工具和指令，而不仅仅是记忆。附录 A 给出了与相关工作的详细比较。

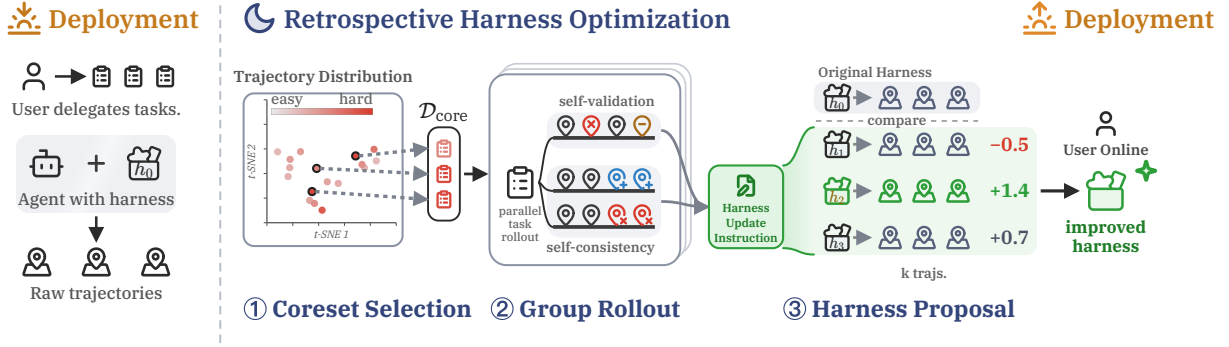


图 2: RHO 流水线。核心集选择通过行列式点过程 (DPP) 挑选一个规模小、难度多样的过往任务子集。群体展开对每个任务重新求解 G 次, 并诊断轨迹内失败 (自验证) 与跨轨迹分歧 (自洽性)。工具提案采样 N 个候选工具, 并保留其展开结果最优于基线的那个。不使用任何真实标签。

3 问题设定

本文将工具 h 定义为智能体可用于解决任务的工具、提示与技能的持久集合。给定任务 t 与工具 h , 智能体可通过推理、行动与观察的循环来尝试该任务。这一多步过程生成轨迹 τ , 记录智能体读取的信息、其思维链、使用的工具以及最终输出。本文将这一执行过程表示为带提示的智能体操作 $\tau = \text{solve}(h, t)$ 。当智能体执行多个任务时, 会产生轨迹数据集 $\mathcal{D} = \{\tau_1, \tau_2, \dots, \tau_n\}$ 。这些轨迹常包含失败实例与可用于改进工具的有用洞见。因此, 本文探究智能体能否回顾性地分析过往轨迹, 以优化其工具并提升未来性能。为量化这一点, 本文定义一个潜在效用函数 $U(t, \tau)$ 来衡量轨迹质量。本文将优化形式化为函数 $\text{optimize}(h, \text{instruction})$, 该函数返回修改后的工具 h' 。目标是找到最优工具 h^* , 以最大化未来任务上的期望效用:

$$h^* = \arg \max_{h'} \mathbb{E}_{t, \tau \sim \text{solve}(h', t)} [U(t, \tau)].$$

问题。然而, 在实践中准确估计该效用函数十分困难。为评估工具的真实效用, 需要一组具有代表性的未来任务验证集, 以及一种计算智能体使用该特定工具时成功率的机制。在本文设定中, 函数 U 是潜在的, 无法直接观测。

本文方法。由于效用 U 是潜在的, 无法直接优化。因此, 本文用自偏好估计器替代这一潜在效用。

Algorithm 1 A single round of RHO. One backbone instantiates every operator (the difficulty judge, solve, optimize, and rank), differing in its inputs and consulting no ground-truth label.

Input: past trajectories $\mathcal{D} = \{(t_i, \tau_i)\}_i$ and harness h_0 , with coreset size k , group size G , candidate count N , and DPP weight θ

Output: updated harness h^*

STAGE 1 CORESET SELECTION

- 1: $r_i \leftarrow \text{judge}(t_i, \tau_i) \quad \forall (t_i, \tau_i) \in \mathcal{D}$
- 2: $\mathcal{D}_{\text{core}} \leftarrow \text{DPP-GREEDY}(\{(t_i, r_i)\}; \theta, k)$

STAGE 2 GROUP ROLLOUT

- 3: **for** $t \in \mathcal{D}_{\text{core}}$ **in parallel do**
- 4: $\{\tau_{t,g}\}_{g=1}^G \leftarrow \text{solve}(h_0, t)$ $\triangleright k \times G$ rollouts
- 5: $\tau_t^{(0)} \leftarrow \tau_{t,1}$ $\triangleright$ fixed baseline rollout
- 6: $I_t \leftarrow \text{rank}_{\text{val}}(t, \{\tau_{t,g}\}) \cup \text{rank}_{\text{con}}(t, \{\tau_{t,g}\})$
- 7: **end for**
- 8: $I \leftarrow \bigcup_{t \in \mathcal{D}_{\text{core}}} I_t$

STAGE 3 BEST-OF- N HARNESS PROPOSAL

- 9: **for** $j = 1, \dots, N$ **in parallel do**
- 10: $h_j \leftarrow \text{optimize}(h_0, I)$
- 11: $\tau_t^{(j)} \leftarrow \text{solve}(h_j, t) \quad \forall t \in \mathcal{D}_{\text{core}} \quad \triangleright N \times k$ re-solves
- 12: $S_j \leftarrow \frac{1}{k} \sum_{t \in \mathcal{D}_{\text{core}}} \text{rank}(t, \tau_t^{(j)}, \tau_t^{(0)})$
- 13: **end for**
- 14: $j^* \leftarrow \arg \max_j S_j$
- 15: **return** h_{j^*} **if** $S_{j^*} > 0$, **else** h_0

具体而言, 我们指示智能体比较同一任务上的多条轨迹, 以计算自我偏好排序。我们定义一个排序函数为 $(\text{rank}, \text{理由}) = \text{rank}(t, \tau_1, \tau_2, \dots, \tau_m)$ 。该函数对给定轨迹产生偏好顺序, 并提供理由解释智能体为何偏好某些执行而非其他执行。下一节详细说明我们如何组织求解、排序和优化操作, 以提升潜在框架效用。

4 回顾性框架优化

我们提出回顾性框架优化 (RHO), 一种仅利用过去轨迹改进框架的自监督方法。

具体而言，我们的流程（图 2）包含三个阶段，即核心集选择、分组展开和 N 选最优框架提议。首先，我们选择过去任务的一个代表性子集来定义优化目标。接着，我们为核集中集中的每个任务采样一组并行展开，并从中提取框架改进信号。最后，我们基于这些信号生成 N 个候选框架，并利用成对自我偏好保留最受偏好的一个。完整算法详见算法 1。

4.1 核心集选择

给定大量历史轨迹，我们需要提取最关键信号以指导测试框架优化。对每条轨迹逐一优化在计算上不可行，且可能因琐碎信号而稀释重要信号。为此，我们首先从完整集合 $\mathcal{D}$ 中选取一个核心集 $\mathcal{D}_{\text{core}}$ ，以代表最需要优化的轨迹，这与基于轨迹的数据估值方法相呼应，该方法通过样本对优化的影响来评分训练样本（Xu et al., 2025）。具体而言，我们要求核心集同时涵盖具有挑战性和多样性的场景。这一要求促使我们的优化在解决最困难问题时覆盖广泛的失败模式。为实现这一目标，我们引入行列式点过程（Determinantal Point Process, DPP）核函数（Kulesza and Taskar, 2012），在满足多样性约束的前提下，按难度对所有历史轨迹进行排序。实践中，我们使用语言模型评判器分析每条轨迹 τ_i ，提取难度分数 r_i 及文本描述。该描述详细说明问题的具体挑战和潜在失败模式。随后，我们计算该描述的嵌入，并使用嵌入间的余弦相似度作为任意两条轨迹 τ_i 和 τ_j 的相似性度量 $S_{i,j}$ 。综合考虑难度分数 r 和轨迹相似性矩阵 S ，我们构建 L-集成核矩阵

$$L = \text{diag}(\tilde{r}) S \text{diag}(\tilde{r}),$$

其中 $\tilde{r}_i$ 是轨迹难度分数 r_i 的缩放版本：

$$\tilde{r}_i = \left(\max(r_i, \epsilon) / \max_j \max(r_j, \epsilon) \right)^\alpha,$$

$$\alpha = \theta / (2(1 - \theta)).$$

利用该核函数 L ，DPP 以与核行列式 $\det(L_Y)$ 成比例的概率选取子集 Y ，并通过参数 θ 调整难度与多样性的相对重要性，即 α 。当 $\theta = 1$ 时，轨迹仅按难度排序；当 $\theta = 0$ （均匀权重）时，仅按相似性多样性排序。采用 $\theta = 0.7$ ，我们选取 k 条轨迹构成核心集 $\mathcal{D}_{\text{core}}$ ，以覆盖困难且多样的失败模式，供后续阶段使用。

4.2 分组展开

受先前工作启发，该工作通过计算相对于一组采样轨迹的优势，以该组作为基线替代学习到的价值模型（Shao 等, 2024），我们通过对每个核心集任务运行 G 个并行智能体求解来生成一组轨迹。随后，智能体比较这些组内轨迹以识别表现不佳的运行。然后，智能体利用组内的对比信号来制定优化测试框架的指令。具体而言，我们从两个维度进行这种自偏好分析。• 自验证（rankval）。该维度考察智能体在每条轨迹内的正确性。智能体根据所需任务和环境观测检查每条轨迹，以确定目标是否被高效达成，利用模型识别自身知识局限性的部分能力（Pan 等, 2025）。在此过程中，它标记错误的工具调用、错误假设和过早停止。这些被标记的方面随后被提取为相对表现不佳运行的改进领域。

- 自洽性（rankcon）。该维度考察智能体的行为在不同轨迹间是否保持一致。由于低自洽性通常表明高不确定性（Wang 等, 2022; Farquhar 等, 2024），我们指示智能体分析轨迹间的矛盾。智能体识别重要的分歧，例如不同的计划、工具序列或最终答案，并生成优化指令以鼓励更一致的行为。这些 rankval 和 rankcon 分析产生 JSON 格式的结构化评估，对于每个任务，它们的并集构成改进指令 $I_t = \text{rankval}(t, \{\tau_{t,g}\}) \cup \text{rankcon}(t, \{\tau_{t,g}\})$ 。因此，我们合并核心集中所有任务的 $\{I_t\}$ ，形成最终的测试框架改进指令。

表 1: 测试框架优化后的留出集通过率。架构列指示每种方法编辑的测试框架表面。 Δ is 相对于同一留出集上 Vanilla Codex 的绝对变化。

Method	Harness	SWE-Bench Pro		Terminal-Bench 2		GAIA-2	
	Architecture	Pass	Δ	Pass	Δ	Pass	Δ
Vanilla Codex	None	0.59	n/a	0.71	n/a	0.29	n/a
Dynamic Cheatsheet (Suzgun et al., 2025)	Skills	0.62	+0.03	0.73	+0.02	0.30	+0.01
ReasoningBank (Ouyang et al., 2025)	Memory	0.61	+0.02	0.73	+0.02	0.28	-0.01
Sleep-time Compute (Lin et al., 2025)	Memory	0.64	+0.05	0.73	+0.02	0.32	+0.03
RHO	Skills+Tools	0.78	+0.19	0.76	+0.05	0.37	+0.08

4.3 最佳 N 框架提案 在获得改进指令后，我们通过这些指令提供给智能体来优化框架。然而，正如先前关于智能体进化的研究所观察到的（Agrawal 等人，2025；Hu 等人，2024；Lee 等人，2026），框架优化本质上是随机的，即使有有效的输入信号，也可能无法可靠地提升性能。为了缓解这一局限性，我们并行采样框架提案，并使用智能体自我偏好进行筛选。这种选择旨在偏向那些改进能够泛化到未来任务的候选框架。具体来说，我们执行 N 次并行优化调用，生成 N 个候选框架，记为 h_1 至 h_N 。随后，我们使用这些候选框架在 k 个核心集任务上获取 N 组新轨迹。对于每个核心集任务，我们通过将每个候选框架产生的新轨迹与原始框架产生的旧轨迹进行排序，来计算智能体偏好分数。我们在核心集上聚合这些分数，以确定每个候选框架的相对优势分数：

$$S_j = \frac{1}{|\mathcal{D}_{\text{core}}|} \sum_{t \in \mathcal{D}_{\text{core}}} \text{rank}\left(t, \tau_t^{(j)}, \tau_t^{(0)}\right),$$

其中 $\tau_t^{(0)}$ 是任务 t 的原始框架轨迹。最后，我们返回具有最大相对优势的候选框架来替换原始框架。仅当其分数严格大于零时，我们才接受此更新（ $S_j > 0$ ）。

5 实验与结果

设置。我们使用 Codex 智能体（OpenAI, 2025）作为回顾性优化的基础框架。具体来说，该智能体使用配置了高推理力度的 GPT-5.5（OpenAI, 2026）。当我们调用 Codex 解决任务时，我们将框架构建为一个可配置的工作区文件夹。该

文件夹包含作为工具的可执行脚本，以及用于技能和指令的文本文件。在所有实验中，我们将核心集大小 k 设为 10。此外，并行轨迹采样和框架提案的数量均设为 3。为了衡量改进效果，我们报告了使用原始 Codex 框架和优化后框架在留出测试集上的通过率。

数据。我们从现有基准数据集中收集历史轨迹。

具体而言，我们将原始基准划分为轨迹集和测试集。随后，我们在轨迹集上运行原始 Codex 智能体，以生成 RHO 所需的轨迹。在此基础上，我们在 SWE-Bench Pro、Terminal-Bench 2 和 GAIA-2 上评估 RHO。SWE-Bench Pro 包含需要仓库级推理和多文件编辑的长程软件工程任务（Deng 等，2025）。Terminal-Bench 2 包含带有可执行评分器的命令行任务（Merrill 等，2026）。GAIA-2 在动态、异步环境中评估用于知识工作的 LLM 智能体（Froger 等，2026）。因此，这三个基准覆盖了软件工程、技术工作和知识工作中广泛的任务类型。在优化过程中，RHO 仅使用历史轨迹及其自身判断；评分器仅用于留出评估。我们在附录 E 中提供了基准和数据划分的详细信息。

5.1 与无反馈方法的比较 基线

我们将 RHO 与三种不需要验证反馈的竞争性框架优化方法进行比较。对于基线，动态备忘单维护有用事实和程序的运行记录（Suzgun 等，2025）。推理库存存储可复用的推理模式，并在推理时检索前 k 个相关条目（Ouyang

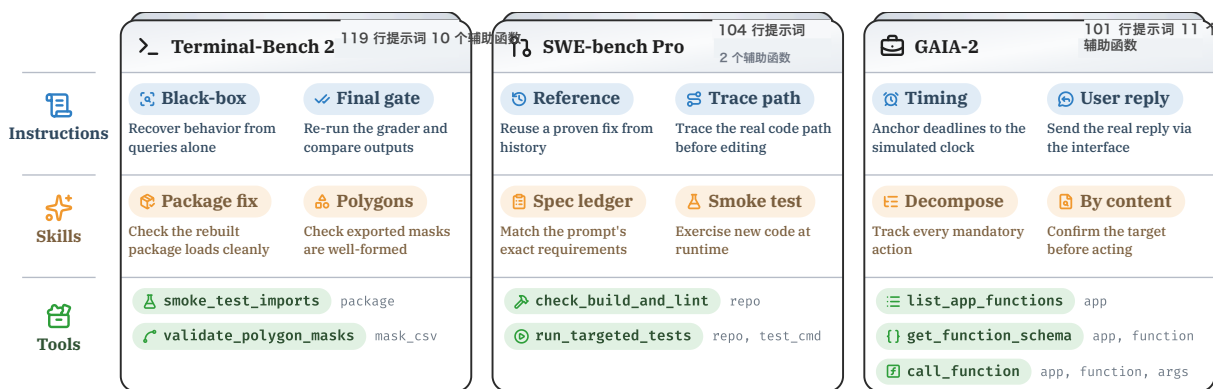


图 3: RHO 在各基准上产生的最高得分框架。指令是与任务无关的程序性规则，技能记录了此前导致失败的评分器或环境特性，工具则是可执行脚本。所示项目具有代表性，每个框架的完整逐字内容见附录 H。

等人，2025)。类似地，睡眠时间计算在离线状态下将过去的轨迹预处理为紧凑笔记，然后将其前置到智能体的上下文中 (Lin 等人，2025)。我们将每个基线适配到我们的数据集和智能体设置，同时保持总智能体调用预算大致固定，以确保公平比较。详细的适配过程见附录 F。如表 1 所示，RHO 在所有三个基准上均带来一致的改进，而基线方法则不然。最值得注意的是，我们在不依赖任何基于验证的评分的情况下，在 SWE-Bench Pro 上取得了 19% 的绝对改进。我们将这一优势归因于 RHO 所实现的更灵活的框架优化。具体而言，智能体可以为框架创建新的工具、技能和指令，而以往的方法主要侧重于记忆系统或基于文本的技能。此外，自我偏好的使用可能有助于这些框架改进的一致性。相比之下，基线方法的性能提升往往较小，且在不同数据集间存在差异。在下一节 (第 5.2 节) 中，我们将考察 RHO 如何修改框架以改进智能体。

5.2 优化后的框架包含什么

图 3 总结并解释了 RHO 优化后生成的新框架内容。在我们的工作中，框架被具体化为一个目录，其中包含用于指令和技能的 Markdown 文件，以及用于工具的可执行脚本。在所有三个基准上，RHO 都为框架添加了多个新技能和工具。这些新增内容中有许多解决了原始框架遇到的典型失败模式。例如，在 SWE-Bench Pro 中，智能体学习到 Go

工具链位于默认路径之外的非标准位置。它还发现，在生成最终差异之前必须剥离 Python 缓存目录，因为不这样做通常会导致补丁无法干净地应用。为了解决这些问题，该代理添加了一个新的 `check_build_and_lint` 工具，用于定位非标准工具链并标记必须排除在补丁之外的生成工件，从而修复了原始轨迹反复遗漏的差异卫生程序。这些示例说明了 RHO 如何通过分析过去的失败，在不同场景中识别有用的工具和技能。

5.3 与验证反馈的比较优化

接下来，我们将 RHO 与 Meta-Harness (Lee 等人，2026) 进行比较。Meta-Harness 是一种验证反馈优化器：提议者发出测试框架编辑，每个候选者在带有真实标签的搜索集上使用真实评分器进行评分，搜索保留在该集合上得分最高的测试框架。为了保持公平比较，我们使用相同的 Codex 代理作为 Meta-Harness 的提议者和求解器。与 RHO 不同，Meta-Harness 需要保留的标签，并且由于它在多轮中演化，因此消耗更多的代理调用。因此，我们在单轮中评估它以匹配我们的计算预算，并在十轮的扩展预算设置中进行评估。在匹配的单轮预算下，Meta-Harness 使用验证分数选择其最佳候选者，但在 SWE-Bench Pro 上仅达到 0.62 的通过率。这显著低于 RHO 实现的 0.78 通过率。将 Meta-Harness 扩展到 10 轮设置将其性能上限提高到 SWE-Bench Pro 上的 0.80。然而，这种更高的性能需要大约三倍于 RHO 的优化阶段计算量 (320 对

表 2: RHO 与 Meta-Harness（一种验证反馈优化器）在 SWE-Bench Pro 上的比较。代理调用是相对于一次 RHO 运行的优化时间预算。保留的通过率与表 1 相同的划分。

Method	Val. labels	Agent calls	SWE-Bench Pro
RHO	none	103 (1.0 $\times$)	0.78
Meta-Harness (1 round)	required	41 (0.4 $\times$)	0.62
Meta-Harness (10 rounds)	required	320 (3.1 $\times$)	0.80

103 次代理调用），更重要的是，仍然依赖于 RHO 不使用的保留标签。

6 讨论

6.1 优化后代理行为如何变化？

尽管 RHO 为智能体创建了新技能和工具，但这些更新通过何种机制使智能体在未来任务中表现更佳，并非一目了然。为探究此问题，我们可视化了工具调用频率以及随智能体执行步数变化的累积成功率。具体而言，图 4 绘制了在给定智能体步数内解决的留出任务的累积比例，并展示了智能体动作组合随时间的变化。我们观察到，三个数据集上的性能提升主要源于需要长时程的任务上成功率的提高。相比之下，增益集中在长时程任务而非步数较少的任务上，这在 SWE-Bench Pro 上最为明显。此外，优化过程改变了智能体的工作模式。因此，优化后的智能体转向更依赖特定类型的动作。例如，在 SWE-Bench Pro 上，智能体验证其工作的频率显著增加。这种主动验证似乎解释了长时程任务上大部分性能提升。在 Terminal-Bench 2 和 GAIA-2 上，智能体通过积极应用新开发的工具来提高准确率。新添加的工具几乎在每个留出任务上都被调用，而在 GAIA-2 上，新的环境助手每个任务大约运行二十次。Terminal-Bench 2 是一个具有启发性的例外——生成的脚本从未被执行，增益反而来自写入指令中的程序化操作手册，这表明 RHO 针对每个环境调整了工具接口，而非总是添加可执行工具。作为对我们

表 3: 最佳 N 工具接口提案与单个采样候选的对比。 $N=3$ 个候选的留出通过率。均值为均匀随机选择下的期望得分，选定为 RHO 部署的候选。

Dataset	Mean	Chosen	Std	Lowest
SWE-Bench Pro	0.79	0.78	0.06	0.73
Terminal-Bench 2	0.74	0.76	0.03	0.71
GAIA-2	0.34	0.37	0.03	0.32

表 4: 在全部三个基准上对诊断步骤的消融。每行在保持提案智能体提示词不变（§ 6.4）的情况下，将完整 RHO 的诊断中某一线索置空后重新渲染。原始轨迹跳过诊断，直接向提案智能体展示原始轨迹。单元格报告留出通过率。

Variant	SWE Pro	TB 2	GAIA-2
Full diagnosis	0.78	0.76	0.37
– self-consistency	0.56	0.75	0.27
– self-validation	0.70	0.73	0.30
Raw trajectory	0.60	0.75	0.29

动作级统计之外，基于长推理链的令牌级归因为长时程智能体行为提供了更细粒度的视角（Pan 等人，2026b）。

6.2 核心集选择如何塑造能力演化？

本文研究了核心集选择如何影响优化过程。为此，我们将基于行列式点过程（DPP）的选择方法与几种消融策略进行了对比。这些变体包括仅按难度选择任务（ $\theta = 1$ ）、纯粹为最大化覆盖度选择任务（ $\theta = 0$ ）以及随机采样轨迹。此外，我们测量了每种选择策略下优化后的框架的最终性能。如图 5 中任务嵌入的 t-分布随机邻域嵌入（t-SNE）投影所示，仅按难度选择任务会导致所选样本聚集在任务分布的狭窄区域。这种聚类现象的产生是因为语言模型将某些类型的任务判定为固有地更难，从而未能将其他任务类型纳入核心集。因此，该策略在优化后未带来有意义的性能提升。类似地，仅优化覆盖度也产生了次优结果。相比之下，随机采样偶尔能选到对优化有用的轨迹。这些发现表明，一种兼顾难度与多样性的核心集选择策略对于提供正确信号以指导框架优化至关重要。

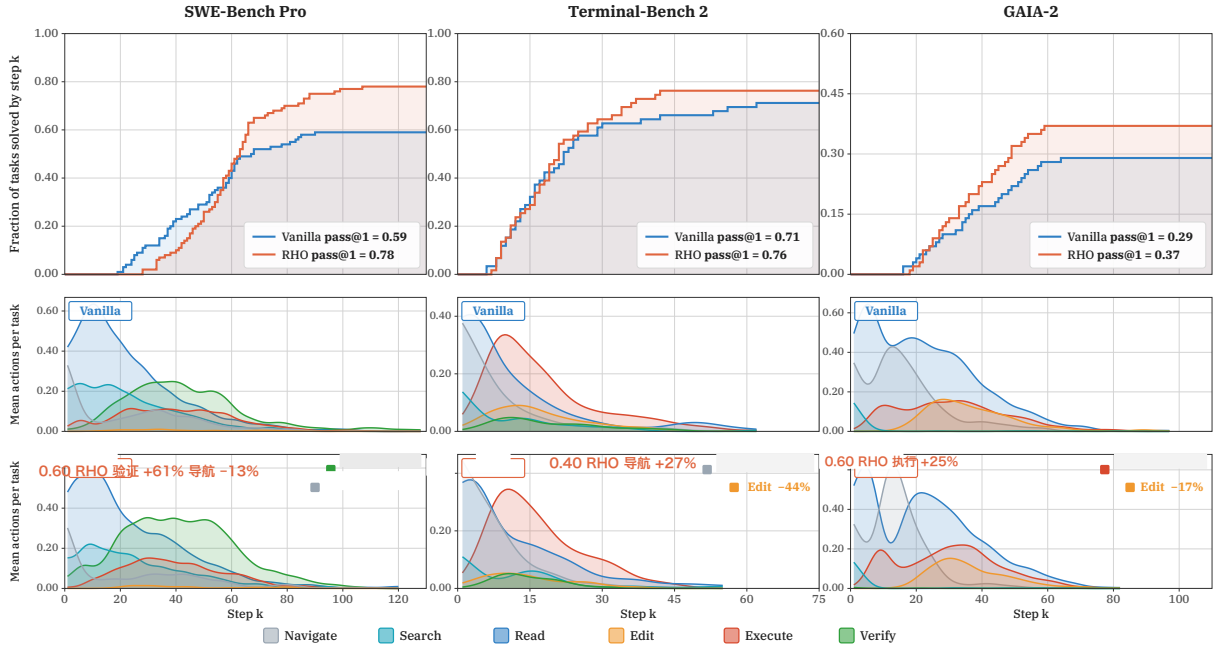


图 4: RHO 后的行为转变。RHO 维持更长的工作会话, 并在 SWE-Bench Pro 上将智能体每步动作组合转向验证, 在 Terminal-Bench 2 和 GAIA-2 上转向执行。

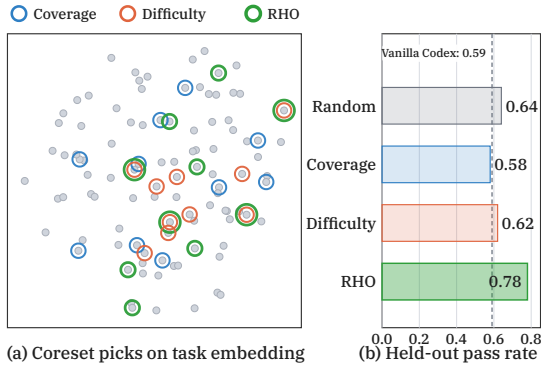


图 5: SWE-Bench Pro 上的核心集选择。(a) 各选择器所选样本在任务嵌入上的分布, 覆盖度选择分散, 难度选择聚集, 而 RHO 的 DPP 兼顾两者。(b) 从各核心集优化得到的框架在留出集上的通过率。仅按难度或多样性甚至不如随机采样, 只有 DPP 的组合达到了最高增益。

6.3 RHO 是否产生一致的框架更新?

由于 RHO 在没有真实标签参考的情况下运行, 我们评估其优化输出在多次运行间是否一致。这种一致性与我们最佳 N 框架提案的整体有效性密切相关。我们检验选择策略是否能可靠地识别出在下游任务上表现最佳的候选框架。在本实验中, 我们测量所有三个生成的候选框架的测试分数, 而不仅仅是最优的。

一、表 3 显示, 生成的测试框架仅表现出中等程度的方差。值得注意的是, 即使得分最低的候选框架, 相较于基线仍能显著提升智能体性能。此外, 最优 N 选择机制可防止部署性能不佳的框架。具体而言, 所选框架在全部三个基准上的得分均高于最差候选框架。同时, 最受偏好的框架并不总是与测试集上得分最高的候选框架一致, 但该选择机制始终能避开最差候选框架。进一步比较每个候选框架的偏好得分 S_j 与其留出集通过率可知, 自我偏好是一种保守的选择器, 而非完美的排序器: 在 SWE-Bench Pro 上, 一个偏好度较低的候选框架实际达到了 0.85, 高于所选框架的 0.78。该机制可靠保证的是安全性, 而非完美排序。由于仅当智能体偏好新框架胜过当前框架 ($S_j > 0$) 时才部署更新, 因此所部署的框架在任何基准上均未低于原始基线; 在 SWE-Bench Pro 和 GAIA-2 上, 所有候选框架均优于原始基线, 而在 Terminal-Bench 2 上, 最差候选框架与原始基线持平 (0.71), 所选框架则达到 0.76。

6.4 回顾性分析有多大贡献?

我们分析了回顾性分析中提取的两种信号的贡献, 即自我

验证与自治性。我们将这些显式信号与一种更直接的方法进行比较，该方法在优化过程中提供原始轨迹，并跳过显式的回顾性分析步骤。此外，我们研究了每个诊断信号对最终性能的单独贡献。

为探究群体展开与诊断阶段的必要性，本文进行了消融实验。具体而言，本文分别移除自验证与自治性信号，随后重新执行优化与评估流程。本文还引入原始轨迹基线。该基线绕过独立的排序分析，直接将原始轨迹提供给优化步骤，要求智能体在单次处理中分析轨迹并提出改进。表 4 显示，移除自治性或自验证信号均会一致地降低各基准上的最终性能。该结果表明，两种信号对于优化框架均至关重要。此外，完整诊断在所有三个基准上均优于简化的原始轨迹基线。原始轨迹基线赋予提案智能体与 RHO 相同的可编辑范围——工具、技能与指令——因此与完整诊断的差距（SWE-Bench Pro 上 0.60 对 0.78）反映了结构化回顾分析的价值，而非更广的编辑范围。该结果表明，尽管单次轨迹分析是可行的，但显式的自验证与自治性信号是必要而非偶然的，能够在各数据集上产生更可靠的改进。最后，表 4 是一项受控消融实验，在保持流水线其余部分固定路径（ $n=1$ ）的同时移除一个诊断线索，因此它衡量的是每个线索的边际贡献；不应将其解读为禁用某个信号的端到端效应。为衡量运行间鲁棒性，本文还针对每种消融策略端到端地重新运行完整流水线两次，使用独立生成的候选集：每次重新运行的配置均保持在其均值的 $\pm 0.01 - 0.03$ 范围内（SWE-Bench Pro 上无自治性为 0.79 ± 0.01 ，无自验证为 0.815 ± 0.025 ， $n=2$ ），且在任何重新运行中性能均未降至受控消融水平。在 $n=2$ 下，这仅具指示性而非结论性，但表明当某个线索缺失时，最佳 N 提案与剩余信号能够进行补偿，因此移除一个组件不会破坏流水线的稳定性。

7 结论

本文提出回顾式线束优化（RHO），将线束改进重新定义为智能体可基于自身过往经验运行的回顾过程，而非由外部真值反馈引导的搜索。其核心思想在于，智能体自身的轨迹已蕴含改进所需的信号，因为重新求解过往任务并比较结果能够揭示线束的失效之处及修复方案。在软件工程、技术工作和知识工作三个领域中，该自监督循环均带来一致的留出集增益，并重塑了智能体的工作方式。上述结果基于代码风格命令行智能体在单轮回顾中取得；将 RHO 扩展至其他骨干模型、智能体框架以及迭代多轮优化，是自然的未来工作方向。我们认为 RHO 是迈向智能体在部署中持续从积累经验中自我改进的一步，而部署场景中标注数据极为稀缺。

局限性

本文提出 RHO，一种无需任何外部评分、仅利用智能体自身过往轨迹的自监督方法，以改进其线束。然而，在缺乏真值反馈的情况下运行存在若干局限性。第一，分组回放会对每个核心集任务进行多次重放，这要求环境能够干净地重置并容忍重复尝试，因此一次性或不可逆任务不在 RHO 所针对的设定之内。第二，RHO 假定智能体能力的相当一部分由可编辑的提示、技能和工具线束所承载；我们的实验覆盖软件工程、技术工作和知识工作，将 RHO 扩展至具有不同线束表面、任务指纹和回放预算的领域仍是未来工作。第三，所有实验均采用单一骨干模型和智能体框架（基于 GPT-5.5 的代码风格命令行智能体）。该方法的信号仅来源于智能体自身轨迹和成对判断，因此本质上并不依赖于这一特定骨干模型，但验证其能否迁移至其他骨干模型和智能体框架仍需进一步实验，我们将其留待未来工作。第四，我们仅评估了单轮回顾优化，并未声称多轮优化中增益会累积甚至保持单调；在连续轨迹批次上研究迭代式多轮 RHO 是未来工作。第五，优化后的

测试框架 (harness) 针对各自的基准环境进行了适配 (例如, 面向 SWE-Bench Pro 的修复验证工具, 以及面向 Terminal-Bench 2 的程序化操作手册), 而非构建一个通用框架; 我们将正式的跨基准迁移研究留待未来工作。第六, RHO 仅将历史轨迹作为其唯一的优化输入; 在开放环境中, 轨迹可能嵌入任务中途注入的对抗性内容 (Liu 等, 2026), 而从受损轨迹中提炼出的框架更新可能会固化此类行为, 从而强化伦理声明中的审计实践。

伦理声明

RHO 根据模型生成的判断来修改持久化智能体行为。如果评估者偏好错误偏好、不安全流程或有偏的行为规则, 这一过程可能会放大这些因素。部署时应保留完整的审计日志, 对敏感的框架编辑需经人工批准, 并在将已接受的框架应用于高风险任务之前, 采用领域特定的安全检查。

可复现性声明

每次运行都会持久化提示、补全、轨迹、诊断、候选框架、框架差异、配置、分数、运行元数据以及留出报告。本文草稿中的数字直接读取自记录的运行报告。与相关工作的详细比较见附录 A, 提示见附录 B, 超参数见附录 C, 完整流程细节见附录 D, 各数据集规范见附录 E, 基线实现见附录 F, 各方法优化阶段的计算成本见附录 G。

致谢

本工作由香港研究资助局基金 RFS2425-1S01 支持, 并在第一作者于微软亚洲研究院实习期间完成。

参考文献

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, and 3 others. 2025. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*.

Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630.

Romain Froger and 1 others. 2026. Gaia2: Benchmarking LLM agents on dynamic and asynchronous environments. In *International Conference on Learning Representations (ICLR)*. ArXiv:2602.11964.

Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.

Alex Kulesza and Ben Taskar. 2012. Determinantal point processes for machine learning. *Foundations and Trends in Machine Learning*, 5(2–3):123–286.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.

Kevin Lin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, and Joseph E. Gonzalez. 2025. Sleep-time compute: Beyond inference scaling at test-time. *arXiv preprint arXiv:2504.13171*.

Minhua Lin, Zhiwei Zhang, Hanqing Lu, Hui Liu, Xianfeng Tang, Qi He, Xiang Zhang, and Suhang Wang. 2026. MemMA: Coordinating the memory cycle through multi-agent reasoning and in-situ self-evolution. *arXiv preprint arXiv:2603.18718*.

Zhichao Liu, Wenbo Pan, Haining Yu, Ge Gao, Tianqing Zhu, and Xiaohua Jia. 2026. WebTrap: Stealthy mid-task hijacking of browser agents during navigation. *arXiv preprint arXiv:2605.08310*.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, and 1 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations (ICLR)*. ArXiv:2601.11868.

OpenAI. 2025. OpenAI Codex. <https://developers.openai.com/learn/codex>. Accessed 2026-08-29.

- OpenAI. 2026. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. 2026. Skill-IO: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2025. ReasoningBank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.
- Wenbo Pan, Shujie Liu, Xiangyang Zhou, Shiwei Zhang, Wanlu Shi, Mirror Xu, and Xiaohua Jia. 2026a. M*: Every task deserves its own memory harness. *arXiv preprint arXiv:2604.11811*.
- Wenbo Pan, Zhichao Liu, Xianlong Wang, Haining Yu, and Xiaohua Jia. 2026b. Towards long-horizon interpretability: Efficient and faithful multi-token attribution for reasoning LLMs. *arXiv preprint arXiv:2602.01914*.
- Wenbo Pan, Jie Xu, Qiguang Chen, Junhao Dong, Libo Qin, Xinfeng Li, Haining Yu, and Xiaohua Jia. 2025. Can LLMs refuse questions they do not know? Measuring knowledge-aware refusal in factual tasks. *arXiv preprint arXiv:2510.01782*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2025. Dynamic cheat-sheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952*.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.
- Zihan Wu, Jie Xu, Yun Peng, Chun Yong Chong, and Xiaohua Jia. 2026. MulVul: Retrieval-augmented multi-agent code vulnerability detection via cross-model prompt evolution. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muenighoff, Defu Lian, and Jian-Yun Nie. 2023. C-Pack: Packed resources for general chinese embeddings. *arXiv preprint arXiv:2309.07597*.
- Jie Xu, Zihan Wu, Cong Wang, and Xiaohua Jia. 2025. LiveVal: Real-time and trajectory-based data valuation via adaptive reference points. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2023. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic “differentiation” via text. *arXiv preprint arXiv:2406.07496*.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2022. Large language models are human-level prompt engineers. *arXiv preprint arXiv:2211.01910*.

表 5：RHO 与先前方法的比较。 右侧区块标记了每种方法是否在三个维度上满足 RHO 的设置。无标签：不使用真实度量或验证集。完整框架：编辑可执行工具和技能，而非仅编辑记忆或提示文本。单次通过：一次性的回顾性通过，而非在线流、迭代验证评分搜索或权重训练循环。满足，● 部分满足，○ 不满足。

Method	Harness architecture	Feedback signal	Cost	Satisfies RHO’s setting		
				Label-free	Full harness	Single pass
<i>Validation-feedback optimization</i>						
OPRO (Yang et al., 2023)	Prompt	Validation metric	Iterative search	○	○	○
DSPy (Khattab et al., 2023)	Prompt + demos	Validation metric	Iterative search	○	○	○
TextGrad (Yuksekgonul et al., 2024)	Prompt	Textual gradient	Iterative search	○	○	○
GEPA (Agrawal et al., 2025)	Prompt	Val. metric + reflection	Iterative (genetic)	○	○	○
ADAS (Hu et al., 2024)	Full agent code	Validation accuracy	25–30 iter.	○	●	○
Meta-Harness (Lee et al., 2026)	Full harness code	Search-set score	~20 iter.	○	●	○
<i>Experience-based self-improvement</i>						
Dynamic Cheatsheet (Suzgun et al., 2025)	Cheatsheet text	Self-judgment	Online stream	●	◐	○
ReasoningBank (Ouyang et al., 2025)	Memory items	LLM-as-judge	Online stream	●	○	○
MemMA (Lin et al., 2026)	Memory entries	Synthetic probe QA	Online (session)	●	○	○
Sleep-time Compute (Lin et al., 2025)	Input context	None	Precompute	●	○	●
SkillIOS [†] (Ouyang et al., 2026)	Skill list	RL reward [†]	RL training	◐	◐	○
RHO (ours)	Tools + skills + instr.	Self-preference	Single pass	●	●	●

[†]SkillIOS's reinforcement-learning reward is an LLM-as-judge self-signal rather than gold labels, but it still requires a large training corpus of task streams, a separate judge model, and weight updates.

A 与相关工作的比较

表 5 将 RHO 与第 2 节讨论的先前方法进行了比较。对于每种方法，我们报告其编辑的框架表面、驱动它的反馈信号以及它产生的成本机制，并标记它是否在三个标准上满足 RHO 的设置，即优化是否无标签、是否编辑完整框架，以及是否作为单次离线回顾性通过运行。验证反馈优化器达到表面轴的完整框架端，其中 ADAS 和 Meta-Harness 重写可执行代码，但该家族的每个成员都使用真实度量来引导搜索并针对其进行迭代。基于经验的方法去除了标签，但它们仅在线管理记忆或技能文本，并保持可执行框架不变，而睡眠时间计算离线预计算上下文但不优化任何内容。因此，每种先前方法最多满足三个轴中的两个，而 RHO 是唯一同时满足所有三个轴的条目。

B 提示

本附录收集了实例化 RHO 五个智能体操作符（§ 3，算法 1，图 2）的提示，即求解、用于核心集选择的难度判断器、诊断分析、优化和排序。提示逐字复制。形式为{name}的占位符在调用时用每个块下描述的值填充。

B.1 求解

每次 `solve(h, t)` 调用将框架和任务实例化到 `harness/` 和 `task/` 下的新工作区中，并将下面的包装指令交给智能体。包装器与任务无关，因为框架目录携带所有任务塑造指导，智能体仅被告知如何读取工作区以及如何交付最终答案。相同的包装器被重用于基线展开和每个候选框架展开，因此求解是唯一变化的输入。

Listing 1 The solve wrapper prompt.

```
Solve the task defined in task/prompt.md, using the information and tools
↔ available under harness/.

The harness (harness/) is a toolkit of resources and guidance that helps the
↔ 智能体求解任务。它可以包含任何类型的文件——辅助脚本、↔ 工件、环境设置、带有相关上下文的
文档，以及
```


↪ workflows to follow.

Workspace layout:

harness/ - read and invoke anything here, but do not modify it
task/ - files for this task, including prompt.md

Steps:

1. Familiarize yourself with the information and available tools in harness/.
2. Read and analyze task/prompt.md.
3. Complete the task. For code repair tasks, modify files directly under
↪ task/repo/.
4. Present your final answer in your last message, in the format prompt.md
↪ 指定（若未指定则为纯文本描述）。

B.2 核心集选择（难度判定器）

难度判定器生成分数 $r_i \in [0, 10]$ 和抽象指纹 ϕ_i ，用于驱动第 4.1 节的 DPP 核心集选择器。它同时接收任务描述以及当前测试框架下一条简短先前轨迹的长度受限摘要。摘要按固定令牌预算截断头部或尾部，并且在展示给判定器之前，任何读取任务预期答案文件的命令都会被清除。判定器被要求使用与任务无关的结构化词汇来保持指纹，以便来自不同代码库的指纹在余弦相似度下仍具有可比性。

清单 2 核心集选择中使用的难度判定器提示。

评估以下软件工程任务的难度，并为其撰写抽象的结构化指纹。你也可以参考下方观察到的智能体运行记录来辅助判断。

Output a JSON object (no markdown fences, no extra text):

```
{  
  "difficulty": <float in [0.0, 10.0]>,  
  "abstract_fingerprint": "<see guide below>"  
}
```

难度等级：

- 0-2: 琐碎（明显的一行修复、外观性改动）。
- 3-5: 中等（局部改动、规范定义明确）。
- 6-8: 困难（多文件、非显而易见的设计、细微缺陷）。
- 9-10: 非常困难（跨领域重构、需要深度推理）。

抽象指纹指南 - 用 3-5 句话（约 60-120 词）描述问题的*形态*，使用适用于任何软件项目的词汇。涵盖：- 失败模式：通常会出现什么问题（部分传播、遗漏边界情况、静默优先级回归、新分支下不变量被破坏等）。- 困难来源：是什么使其困难或容易（分散的不变量、模糊的规范、紧密耦合、纯局部算术等）。- 技术复杂度：范围（单函数/多文件/跨模块/架构级）、推理深度（局部/上下文/全局不变量追踪）、变更类型（缺陷修复/功能/重构/回滚）。

不要提及：仓库、产品、公司、框架或库名称；文件路径；函数、类、配置或变量名；与特定代码库相关的领域特定名词。只使用抽象的结构化编程词汇（不变量、优先级、边界、状态协调、传播、顺序、契约等）。

观察到的智能体运行包含具体文件路径、库名称和工具输出。同样将这些抽象出来——指纹描述的是形态，而不是智能体恰好运行的具体代码库。

The observed run is a single noisy sample, not ground truth. Do not

不要仅仅因为智能体在一次尝试中似乎成功就降低难度，也不要仅仅因为一次尝试在引导问题上挣扎就提高难度。将轨迹视为调整你对任务难度和失败模式先验的证据；相对于任务描述本身所暗示的内容来权衡它。

一个良好抽象的指纹示例：“一个多文件重构，其困难来自于在几个独立演进的模块之间保持单一共享不变量的一致性；典型的失败模式是部分传播，即一个调用点采用了新契约，而另一个调用点静默地保留旧契约，产生一个仅在特定输入顺序下才显现的潜在缺陷。规范模糊性低，但推理必须是全局的——每个站点的变更在机械上很小，但需要通过调用图追踪契约。”

```
Task:
---
{query}
---

Observed agent run (under the current harness):
---
{trajectory_digest}
---
```

{query} 是自然语言任务描述。{trajectory_digest} 是同一任务的一条先前轨迹经过清洗、首尾截断后的摘要。该难度值是进入 DPP 核函数的得分 $r_i \in [0, 10]$ ，在 § 4.1 中按 $r_i/10$ 进行归一化，指纹被嵌入为单位向量 x_i ，从而定义相似度矩阵 $S = XX^T$ 。

B.3 诊断

诊断提示实现了 $I_t = \text{rank}_{\text{val}}(t, \{\tau_g\}_{g=1}^G) \cup \text{rank}_{\text{con}}(t, \{\tau_g\}_{g=1}^G)$ (§ 4.2)。工作区向智能体呈现原始任务、每次运行共享的测试框架以及 G 个运行目录。智能体执行五步工作流程，包括逐轨迹检查、失败模式分析（自验证）、跨轨迹分歧分析（自洽性）、一个高层框架改进方向，以及一个严重性评分，该评分同时作为下游的软注意力权重。结构化 JSON 输出将每个字段绑定到固定槽位，以便优化器按严重性进行关注。

Listing 3 The diagnosis prompt.

```
Analyze three solve trajectories for the same task.

The harness (harness/) is a toolkit of resources and guidance that helps the
  ↪ 智能体解决任务。它可以包含任何类型的文件——辅助脚本、  ↪ 工件、环境设置、带有相关上下文的
  文档，以及  ↪ 要遵循的工作流程。

Workspace layout:
task/                - the original task. Read task/prompt.md to understand the
  ↪ question.
harness/             - the shared harness used by all three trajectories.
trajectory_0/         - first solve attempt. Contains events.jsonl,
  ↪ final_message.txt, and workspace_diff/.
trajectory_1/         - second solve attempt. Contains events.jsonl,
  ↪ final_message.txt, and workspace_diff/.
trajectory_2/         - third solve attempt. Contains events.jsonl,
  ↪ final_message.txt, and workspace_diff/.

Your job is to analyze and evaluate the three trajectories. Follow this
  ↪ workflow:

## Step 1: Inspect each trajectory
```

```

For each of trajectory_0, trajectory_1, and trajectory_2:

1. Inspect final_message.txt and events.jsonl to understand the action and
   ↳ decision process.
2. Evaluate whether the trajectory accurately and efficiently completed the
   ↳ 任务。3. 如果轨迹准确完成了任务，则将 successful 设为 1，
   ↳ otherwise set it to 0.
4. In quality_analysis, note what evidence, files, tools, or reasoning steps
   ↳ 轨迹所依赖的内容，以及是否存在不必要的工作、遗漏的 ↳ 信息、误导性证据或错误决策。

## Step 2: Analyze failure modes

If all three trajectories accurately and efficiently completed the task, this
   ↳ 部分可以简短。否则，分析一条或多条轨迹 ↳ 失败或表现不佳的原因。使该分析忠实且可操作。 ↳ 以
   ↳ 轨迹实际行为为依据。

## Step 3: Analyze inconsistency

Compare the three event sequences and final answers. Identify whether there are
   ↳ 它们之间的不一致之处：轨迹在何处以及为何产生分歧，以及 ↳ 这些差异如何影响行为。

## Step 4: Summarize harness improvement direction

Suggest one high-level, simple direction for improving the harness. This should
   ↳ 是基于轨迹分析得出的总体改进方向，而非 ↳ 详细的编辑计划，也不是针对特定任务的硬编码修复。

## Step 5: Assign severity

Set severity to a float from 0.0 to 1.0 for how strongly this task should
   ↳ 影响下一次框架优化：

- 0.0: 无实质问题；所有轨迹均准确回答。
   ↳ efficiently.
  - 0.1-0.3: 轻微低效或较弱顾虑；不应仅据此进行优化。
- 0.4-0.7: 成功与失败混杂、不一致，或存在合理的框架缺陷。
   ↳ is not.
- 0.8-1.0: 明显失败、信息缺失，或高置信度的框架问题。

## Output format

Your final message must be exactly one JSON object (no markdown fences, no
   ↳ 其他文本)：
{ "task_id": "<任务标识，若可从任务提示或上下文中获取>", "严重程度": 0.0, "trajectory_analyses": [ { "轨迹":
"trajectory_0", "成功": 1, "quality_analysis": "<对该轨迹是否成功的忠实分析>"

   ↳ completed the task accurately and efficiently>",
     "issues": "<any missed information, misleading evidence, inefficiency, or
   ↳ incorrect decision; empty string if none>"
   },
   {
     "trajectory": "trajectory_1",
     "successful": 1,
     "quality_analysis": "<faithful analysis of whether this trajectory
   ↳ completed the task accurately and efficiently>",
     "issues": "<any missed information, misleading evidence, inefficiency, or
   ↳ incorrect decision; empty string if none>"
   },
   {
     "trajectory": "trajectory_2",

```

```

    "successful": 1,
    "quality_analysis": "<faithful analysis of whether this trajectory
    ↪ 准确且高效地完成了任务>", "问题": "<任何遗漏的信息、误导性证据、低效或 ↪ 错误决策；若无则为空字符串>"
  }
],
"failure_mode_analysis": "<actionable analysis in markdown of why any
    ↪ 轨迹失败或表现不佳；若无失败则简述>", "inconsistency_analysis": "<以 Markdown 格式对轨迹在何处及为何
    产生分歧进行根因分析 ↪ 轨迹产生分歧，以及这如何影响质量>", "harness_improvement_direction": "<一个高层
    次、简单的改进方向
    ↪ improving the harness>"
}

```

轨迹目录包含完整的事件流、智能体的最终消息以及由执行过程产生的任何工作区差异。表 4 中的 — 自验证和 — 自洽性消融实验移除了相应步骤和相应输出字段，而周围的脚手架保持不变。

B.4 优化

优化提示实现了 $h_j = \text{优化}(h_0, \{I_t\}_{t \in \mathcal{D}_{\text{core}}})$ (§ 4.3)。编辑智能体被授予对测试框架的全新副本的写权限，以及一个按严重程度排序的逐任务诊断目录。指令将严重程度视为软注意力权重，要求在任何编辑之前进行跨任务模式匹配，并阻止针对特定任务的硬编码修复。每个 N 候选方案均使用相同提示独立采样，底层智能体的随机性提供了多样性。

Listing 4 The optimize prompt.

Based on the per-task diagnoses in diagnoses/, analyze and optimize the current
 ↪ 测试框架/ 以提升未来任务上的性能。"更好的性能" ↪ 意味着智能体的最终答案更直接、更正确地回答了
 ↪ 每个任务所问的内容，且浪费的步骤更少。

The harness (harness/) is a toolkit of resources and guidance that helps the
 ↪ 智能体解决任务。它可以包含任何类型的文件——辅助脚本、↪ 工件、环境设置、带有相关上下文的
 文档，以及 ↪ 要遵循的工作流程。

Workspace layout:

```

harness/      - the current harness; you may directly modify it
    ↪ (add/remove/edit any files)
diagnoses/    - one subdirectory per task, each containing:
    - diagnosis.md - structured trajectory analysis, severity, failure modes,
    ↪ inconsistency analysis, and a high-level harness improvement direction
    - prompt.md   - the original task for context

```

Steps

1. Read each diagnosis in diagnoses/task_XXXX/diagnosis.md and the
 ↪ corresponding prompt.md.
2. Use Severity as a soft attention weight from 0.00 to 1.00, not as ground
 ↪ truth. Higher severity means the diagnosis should influence optimization
 ↪ more strongly.
3. Look for patterns across diagnoses - are multiple tasks failing for similar
 ↪ 原因? 4. 优先处理高严重性且反复出现的失败模式以及高严重性且反复出现的
 ↪ inconsistency root causes.
5. Low-severity tasks usually should not cause a harness edit by themselves
 ↪ unless the same issue motif recurs across tasks.
6. Make surgical improvements to harness/ that address the high-level diagnosed
 ↪ issues.

When done, send the changes and your rationale as your final message. If you

未做任何更改，请说明当前测试框架为何已足够。

每个 diagnoses/task_XXXX/子目录存放序列化诊断 JSON 渲染为 Markdown 的结果，以及原始任务提示。子目录按严重程度降序索引，使智能体首先遇到影响最大的诊断。

B.5 成对排序

排序提示实现了 $\text{rank}(t, \tau_a, \tau_b) \in [-10, 10]$ ，用于 N 选优接受（§ 4.3 和算法 1）。评估器看到任务、两个测试框架和两条轨迹。候选轨迹显示为 trajectory_A，基线显示为 trajectory_B，编排器对返回的整数取反，使标量分数始终以基线 $\rightarrow$ 候选为方向，无论呈现顺序如何。先呈现候选可减少我们在试点运行中观察到的后选项偏好偏差。以下评分标准使用 $[-10, 10]$ 中的整数刻度，下游仅使用分数的符号和相对大小（§ 3）。

Listing 5 The rank prompt used in Best-of- N acceptance.

Analyze the difference in performance between harness A and harness B on the
 $\hookrightarrow$ 同一任务。你将看到一个 task/目录（包含 prompt.md）和两个 $\hookrightarrow$ trajectory_*/目录（每个包含 events.jsonl 和 $\hookrightarrow$ final_message.txt）。生成一个 JSON 对象，在 $\hookrightarrow$ 从-10 到+10 的整数刻度上对 A 到 B 的转变进行评分，并附简短理由。

The harness (harness/) is a toolkit of resources and guidance that helps the
 $\hookrightarrow$ 智能体解决任务。它可以包含任何类型的文件——辅助脚本、 $\hookrightarrow$ 工件、环境设置、带有相关上下文的文档，以及 $\hookrightarrow$ 要遵循的工作流程。

Scoring rubric:

- +10: A 到 B 是从不可接受到优秀的转变；B 的轨迹是 $\hookrightarrow$ 高效的，且其答案正确。

- 0: A 和 B 表现相当，或无法确定哪个更好
 $\hookrightarrow$ better.

- -10: A 到 B 是严重退化；B 的轨迹效率低下，且其
 $\hookrightarrow$ answer is wrong.

Workspace layout:

task/	- the original task (prompt.md is the question)
harness_A/	- the harness used by trajectory A
harness_B/	- the harness used by trajectory B
trajectory_A/ $\hookrightarrow$ answer)	- trajectory from harness A (final_message.txt is the
trajectory_B/ $\hookrightarrow$ answer)	- trajectory from harness B (final_message.txt is the

Evaluation steps

1. Read task/prompt.md to understand what the task requires.
2. Read and compare trajectory_A and trajectory_B.

Your final reply must be exactly one JSON object (no markdown fences, no other
 $\hookrightarrow$ text):

```
{"value": <integer in [-10, 10]>, "rationale": "<one-sentence rationale>"}
```

算法 1 中的候选得分 S_j 将该整数（含上述方向翻转）在核心集 $\mathcal{D}_{\text{core}}$ 上取平均，且仅当 $S_j > 0$ 时才接受该候选，否则框架保持在 h_0 。

C 超参数与基础设施

表 6 列出了我们实验中使用的超参数和基础设施设置。正文（第 3 节和第 5.1 节）中已报告的值在此处为方便起见重新列出，之前未说明的值在此处引入。目标是提供一份可审计的规范，每个参数一行，每个单元格一个值，不隐含任何默认值。求解器墙钟时间和评分的数据集特定覆盖在附录 E 中推迟说明，每个智能体角色的提示文本在附录 B 中。表 6 中的若干选择需要简要说明。我们设置 $G = 3$ 是因为三次展开足以让诊断提示暴露跨轨迹分歧，而使用更大的 G 进行试点会线性增加成本，却不会增强诊断信号。我们将 DPP 的权重设为 $\theta = 0.7$ ，使难度优先于多样性，其原则是：一个未被选中的简单任务所携带的优化信号少于一个冗余的困难任务。具体而言， θ 通过难度权重

$$\hat{r}_i = (\max(r_i, \epsilon) / \max_j \max(r_j, \epsilon))^\alpha \quad (\text{其中 } \alpha = \theta / (2(1 - \theta)))$$
 进入，因子 2 抵消了难度项在 $\log \det K$ 中的双重出现，使得 θ 和 $1 - \theta$ 直接对难度项和多样性项加权。我们将接受门限设为严格正而非非负，因为成对自偏好是一个有噪声的估计器，偏向变化的平局打破会在没有预期收益的情况下增加退化风险。最后，我们对求解器、优化器和排序器使用相同的 Codex gpt-5.5 骨干，因为将求解器与评判器解耦会引入混淆因素，导致测得的留出增益可能归因于更强的评判器而非 RHO 本身。上文提到的按数据集配置覆盖在附录 E 中，基线特定配置在附录 F 中。

D 流水线实现细节

本附录记录了支配 RHO 行为但未出现在 § 4 或算法 1 中的设计选择。方程和算子签名由正文固定，以下内容为其可执行的机制。

D.1 框架表示与挂载

工具包（Harness）是一个无固定模式的文件目录，其中散文指令、可执行脚本和结构化配置并排放置。

在每次操作符调用时，工具包被物化为智能体工作目录的一个子目录，智能体以读取任务文件的方式读取它，而不是作为系统提示前缀。求解操作符按惯例以只读方式挂载工具包，而优化操作符挂载一个具有写权限的新副本，并接受智能体在文件系统上留下的任何内容。工具包按内容进行比较，因此如果优化返回与输入相同的工具包，该候选方案将被视为无操作并在评估前丢弃。这使工具包表面保持工具无关性，并允许优化器扩展三种文件类型中的任何一种而无需表示变更。

D.2 角色分离与工作区隔离

同一骨干网络执行求解、诊断分析（ $\text{rank}_{\text{val}}/\text{rank}_{\text{con}}$ ）、优化和排序。角色分离通过工作区内容实现，而非改变骨干网络，其中每个操作符在一个仅包含其应见输入的全新工作区中运行。求解看到任务文件和当前工具包。诊断分析看到任务和该任务的一组轨迹。优化看到工具包目录、诊断指令以及促成这些指令的轨迹回放。最后，排序看到任务、两条轨迹以及两个候选工具包目录并排。因此，按角色消融相当于交换提示和输入，而非模型，这使操作符可直接比较。

D.3 排序、解析与失败处理

成对排序对每个（任务，候选）对调用一次排序。我们首先呈现候选轨迹，然后呈现基线轨迹，再对解析出的标量取反。这种交换是成对评判器中位置偏差的标准缓解措施，无需每次调用随机化顺序。评判器被限制为返回 $[-10, 10]$ 中的单个整数并附一句理由，我们不重试。任何解析或执行失败都确定性地产生零。这使排序成为严格的悲观主义者，因为静默失败会将均值拉向拒绝阈值而非远离它。

D.4 诊断与排序输入

诊断分析消耗一个任务的完整 G 次回放组，并产生一条带有严重性权重的文本指令 I ，而排序严格成对。我们不会将

表 6: RHO 的超参数与基础设施。正文中出现的参数以 $\dagger$ 标记，此处仅为完整性而重述。

Parameter	Value	Description
<i>Backbone agent</i>		
Model †	Codex gpt-5.5	Shared across solve, optimize, and rank.
Reasoning effort †	high	Applied to all roles uniformly.
Sampling temperature	provider default	Codex reasoning models do not expose temperature.
Provider	Cloud-hosted gpt-5.5 (Codex CLI)	Single hosted endpoint.
<i>Coreset selection</i>		
Selector	DPP, greedy MAP	(Kulesza and Taskar, 2012).
Coreset size $k^\dagger$	10	Train tasks.
DPP weight $\theta^\dagger$	0.7	Difficulty/diversity tradeoff.
Score floor ϵ	0.1	Lower bound on normalized difficulty.
Judge model	Codex gpt-5.5, high	Same backbone as the solver.
Trajectory-digest budget	10,000 BPE tokens, head/tail truncation	Ground-truth-revealing commands scrubbed.
Fingerprint embedding	BAAI/bge-large-en-v1.5, 1024-d	Executed locally (Xiao et al., 2023).
<i>Group rollout</i>		
Rollouts per task G	3	Parallel solves under h_0 per coreset task.
Solve wall-clock timeout	900 s	Dataset overrides in Appendix E.
Diagnosis prompt	see Appendix B	Self-validation + self-consistency cues.
Severity range	[0, 1]	Soft attention weight, not a hard threshold.
<i>Best-of-N harness proposal</i>		
Candidates $N^\dagger$	3	Parallel optimizer samples.
Optimizer prompt	see Appendix B	Conditioned on $\{I_t\}_{t \in \mathcal{D}_{\text{core}}}$.
Ranking timeout	300 s per pairwise call	Per rank(t, τ_a, τ_b).
Acceptance threshold	$S_j > 0$ (strict)	Mean pairwise score over the coreset.
<i>Infrastructure</i>		
Agent-call concurrency	10 concurrent calls (cap 30)	Parallelism for Table 1 runs.
Rounds per experiment	1	Unless stated otherwise.
Persistence	full logs to disk	Prompts, completions, trajectories, diagnoses, candidate harnesses, diffs, scores, run summaries.

将结果归入布拉德利-特里评分以供评估。对于每个候选方案，我们将该候选方案在任务中的执行轨迹与从原始组中抽取的固定基线轨迹进行配对，并在所有候选方案中保持该基线不变，从而使比较始终锚定于同一参照。其结果是 $G > 1$ 仅作为诊断工具进入流程，因为它通过跨轨迹不一致性强化了指令 I ，但并不参与决定哪个候选方案胜出。

D.5 优化器动作空间

优化是一个代码代理调用，而非受约束的文本重写器。它将物化的测试框架视为文件系统，可以添加、删除或修改其中的任何文件。我们将诊断指令按严重程度降序排列交给它，严重程度以 [0, 1] 区间内的软注意力权重形式呈现，而非硬性优先级队列，因此代理仍可合并或跳过相互冲突的建议。我们从不从代理的最终消息中解析差异，

当调用返回时，新的线束状态将从目录本身恢复。这保持了动作空间与表示的一致性，即任何适合目录的内容都是有效的编辑。

D.6 接受门与空操作

仅当最佳候选在核心集上的平均成对得分严格为正时，更新才会被接受；平均得分为零则被拒绝。在门控之前，若优化器失败、调用超时或生成的测试框架与输入完全相同，候选将被丢弃。若所有候选均被丢弃，或所有存活候选的得分均为 $S_j \leq 0$ ，RHO 将保持测试框架不变。这种不更新行为属于结果的一部分，而非工具故障。在图 5 中，核心集未能暴露有效弱点的选择器（纯覆盖率、纯难度）使保留准确率接近 Vanilla Codex 基线。

D.7 持久性

我们持久化输入测试框架标识符，以及由各算子（求解、诊断分析、优化和排序）产生的轨迹，均以标识符区分。此外，我们持久化诊断指令、候选测试框架标识符及其相对于输入的差异、每个候选对的成对得分、平均成对得分以及接受标志。每条存储的轨迹都携带其完整事件流、最终消息、工作区差异和墙钟时间。这些记录使得下游审计、重新评分和消融成为可能，而无需重新运行智能体。

E 数据集规范

本附录记录了我们从每个上游基准中获取的内容、如何划分、智能体在求解时看到的内容以及评分如何运行。固定的上游提交使得划分和评分器可以从干净的检出中复现。

E.1 SWE-Bench Pro

SWE-Bench Pro (Deng 等人, 2025) 是我们套件中的长周期软件工程基准。任务只有在多文件补丁通过上游测试集时才能解决，因此失败模式是具体的，并且可以追溯到测试框架的仓库约定和构建命令。来源。我们从 Hugging Face 加载 ScaleAI/SWE-bench_Pro 的测试划分。评估脚本和每个实例的 Docker 镜像来自官方 scaleapi/SWE-bench_Pro-os 仓库，固定到特定提交。¹

划分。行按（种子，instance_id）的 SHA-256 哈希排序，种子固定。前 100 行构成训练池，接下来的 100 行构成留出测试池，其余行未使用。我们在留出测试池上报告结果。

求解接口。每个任务生成一个描述问题的提示文件，以及上游仓库在其基础提交处的新克隆，挂载在工作区中。智能体就地编辑仓库文件，不注入额外工具。评分。我们通过将智能体的工作区编辑重新应用到新检出并获取 git diff -binary 来提取智能体的补丁。如果工作区路径失败，我们回退到扫描智能体最终消息中的围栏差异块。二进制块和自动生成的路径（依赖、缓存和

构建目录）在评分前被剥离。补丁随后在 SWE-Bench Pro 作者分发的官方逐实例 Docker 镜像内应用，并调用官方运行与解析脚本。任务通过当且仅当每个 FAIL_TO_PASS 和 PASS_TO_PASS 测试正确解析。每个任务的 Docker 墙钟时间预算为一小时。

E.2 Terminal-Bench 2

Terminal-Bench 2 (Merrill 等人, 2026) 包含可执行的命令行任务。失败意味着错失奖励，而非错失约定，因此该测试框架主要保证代理检查状态和链接命令方式的一致性。

来源。我们使用上游 Terminal-Bench 2 仓库的固定提交²，共得到 89 个任务。

划分。任务按照与 SWE-Bench Pro 相同的种子方案进行哈希排序。前 30 个构成训练池，其余 59 个构成留出池。

求解接口。每个任务启动一个全新的 Docker 容器，其镜像由任务声明。容器带有清理标签以及每个任务的 CPU 和内存限制。代理的主机工作区被绑定挂载到容器内部。求解提示指示代理在主机上编写 shell 脚本并在容器内执行。当任务声明的代理超时时间到达时，墙钟看门狗终止容器。

评分。我们在仍在运行的容器内，按照其声明的验证器超时时间运行任务的上游测试套件。验证器将奖励值 0 或 1 写入已知路径。任务通过当且仅当奖励值等于 1。我们不应用难度过滤，因此保留的样本涵盖了上游的难度混合。

数据完整性。求解提示指示智能体不要读取测试目录、上游解决方案或验证器日志。这是提示级别的约定，而非沙箱限制。我们记录此约定以保证透明度，因为一个足够对抗性的智能体可以读取验证器并操纵奖励。我们依靠保留池和表 1 中的跨基准一致性来检测此类行为，并且我们尚未观察到此类行为。

E.3 GAIA-2

GAIA-2 (Froger 等人, 2026 年) 与两个编码基准的不同之处在于，环境独立于智能体而演化。因此，测试框架层必须在部分观测和异步事件下编码行为，而不仅仅是稳定的构建约定。

¹ 提交 0c64e26f00b9。

² 提交 53ff2b87d621。

部分观测和异步事件，而不仅仅是稳定的构建约定。

来源。我们加载了 Hugging Face 上 GAIA-2 发布版本的验证集分割，采用 mini 配置，得到 200 个场景。每个场景附带一个异步事件流和一个上游写操作验证器。

分割。场景按照相同的种子方案进行哈希排序。前 100 个构成训练池，其余 100 个构成留出池。我们在留出切片上报告结果。

求解接口。每个场景的工作区包含一个任务提示、一个工具调度器和一个工具目录。智能体每次调用通过单个 shell 命令调用工具。调度器将每次调用转发给运行上游环境的边车进程，该进程推进模拟时间并重放预定事件。场景的真实状态完全由边车持有，智能体从不读取它。

评分。当智能体完成时，边车调用场景的上游验证器。评判大语言模型与我们其他地方使用的 Codex gpt-5.5 骨干相同，路由默认指向相同的 Azure Foundry 端点。当且仅当上游验证器报告成功时，场景通过。

环境修改。对上游环境的两处修改会影响报告的数字，必须予以披露。首先，上游每回合一次 send_message_to_user 调用的上限提高到四次，因为原始上限会惩罚那些本可以正确完成任务的冗长智能体。其次，三个可选的评判放宽开关（事件过滤、宽松的按应用界面评判和琐碎文件系统读取过滤）可作为消融轴使用，但在表 1 报告的实验中全部禁用。

F 基线实现

每个基线都在相同的 Codex gpt-5.5 骨干上以高推理努力运行。仅轨迹家族（动态备忘单、推理库和睡眠时间计算）共享 RHO 的核心集选择预算，即 10 个训练任务和 3 个候选。基线仅在离线阶段持久化的内容以及求解器如何消费这些内容上有所不同。本附录记录了每一个。

F.1 动态备忘单 (Dynamic Cheatsheet) (Suzgun 等人, 2025)

动态备忘单维护一份不断更新的可复用事实与流程清单，这些内容从过去的

轨迹中收集而来。

持久层。一个单独的 Markdown 文件位于测试框架内部，其结构为 <memory_item> 块，每个块包含描述、示例和引用计数。

离线阶段。一个策展代理按顺序遍历选定的训练任务。对于每个任务，它读取当前备忘单和求解记录，并就地重写备忘单。

在线使用。备忘单是测试框架的一部分，求解器将其作为常规框架查找的一部分读取。不会注入额外的提示前缀。

忠实性。上游动态备忘单在每个任务后更新备忘单，因此下一个任务的求解器会看到更新。我们的默认配置在调用策展器之前对所有选定的训练任务进行求解，然后提交一份备忘单。与上游逐示例更新相匹配的单任务配置可作为敏感性设置使用，而表 1 使用默认配置。

F.2 推理库 (ReasoningBank) (Ouyang 等人, 2025)

推理库存储从过去轨迹中提取的可复用推理模式，并在推理时按需检索。

持久层。一个 JSONL 格式的条目库，每个条目包含标题、描述、正文和预计算的嵌入。该库位于测试框架目录之外，因为求解时的访问是通过相似性检索进行的，而非通过框架物化。

离线阶段。对于每个选定的训练任务，我们对其进行求解，询问评判者轨迹是否成功，然后运行提取提示，从轨迹中提炼可复用的推理模式。提取的条目被追加到库中，并在本地计算嵌入。

在线使用。在每次留出求解时，我们使用相同的编码器对任务描述进行编码，按余弦相似度检索前 n 个条目，将其渲染为记忆前言，前置到求解指令中。前 n 固定为 1。

忠实性。我们将上游的 Gemini 嵌入替换为 BAAI/bge-large-en-v1.5 (Xiao 等人, 2023)，这是一个 1024 维的本地执行的句子编码器，因此所有使用检索的基线都共享相同的编码器、维度和存储后端。这消除了记忆库质量与嵌入提供者质量之间的混淆。

F.3 睡眠时间计算 (Lin 等人, 2025)

睡眠时间计算将过去的轨迹预处理为紧凑的笔记，在推理时前置到智能体的上下文中，遵循 Letta 实现 (Packer 等人, 2023)。

持久层。一组有界的 Markdown 记忆块存储在测试框架内部，使用结构化编辑工具（插入、替换、反思、完成）进行编辑，这些工具镜像了上游 Letta 的语义。

离线阶段。一个睡眠时间智能体遍历选定的训练任务。对于每个任务，它读取任务的提示和轨迹，然后发出一系列记忆编辑工具调用来更新测试框架的记忆块。系统提示是上游 Letta 系统提示，逐字使用。

在线使用。求解器将记忆块作为测试框架的一部分读取，就像读取任何测试框架文件一样。

范围。编辑在任务之间累积，睡眠时间智能体在任务之间针对一个不断演化的记忆状态进行工作。我们在训练任务之间不重置记忆。

F.4 元测试框架 (Lee 等人, 2026)

元测试框架 (Meta-Harness) 是一种验证反馈参考方法，其中元代理重写测试框架，并根据带标签的验证集对重写结果进行评分。

持久层。一系列候选测试框架目录，与记录每个候选验证通过率的搜索历史配对。

离线阶段。外层循环在提议者与评估者之间交替：提议者读取搜索历史并生成新的测试框架，评估者使用数据集的地面真值评分器在搜索任务集上对提议的框架进行评分。我们重新实现的默认预算为 20 次外层迭代 $\times$ ，每次迭代 3 个候选 $\times$ ，每个任务 2 次求解试验，远高于我们给予 RHO 的预算；上游报告在 20 次迭代中评估了约 60 个候选框架。

验证反馈足迹。提议者从历史中读取每个候选的逐任务得分、平均得分和通过率，评分器信号直接塑造下一个提议。这是验证反馈家族的标志，也是元测试框架与仅基于轨迹的基线不可比较的轴。

匹配预算配置。对于表 2，我们运行外层循环一次迭代，包含 3 个候选和每个任务 1 次求解试验，因此候选数量与 RHO 的 $N = 3$ 相匹配。验证-

评分成本即表 2 所报告的内容。共享配置。所有四个基线和 RHO 对于给定数据集都从相同的空测试框架开始，使用相同的选定训练任务核心集，并使用相同的评分器在相同的留出划分上进行评分。唯一有意变化的轴是离线阶段持久化的内容以及求解器如何消费它。元测试框架额外消费验证评分，这正是表 2 所测量的轴。

G 优化阶段计算成本

本附录记录了 RHO 及基线的优化阶段计算成本，相关结果报告于表 1 和表 2。我们仅报告数值，不主张效率优势。在匹配核心集预算 ($k = 10, N = 3$ 的仅轨迹方法中，RHO 与睡眠时间计算和动态备忘单处于同一数量级，且比推理库更昂贵。匹配预算的元框架配置是唯一成本性质不同的基线，其上游默认预算将更大，如文末所述。

G.1 核算范围

每种方法在离线阶段都会发出两类模型调用，并在评估时执行相同的逐任务求解。我们分别计数，因为它们经由不同后端且单位成本不同。

代码执行器 (智能体) 调用。一次代码执行器命令行子进程完整运行即为一次智能体调用。每次调用会持久化一个轨迹目录，记录角色（求解、诊断、优化、秩之一）、框架标识、任务标识、墙钟时长及退出状态。我们通过统计这些目录获得各方法的总数。

辅助大语言模型调用。对同一 gpt-5.5 主干网络发出的一次直接应用程序编程接口调用，在 Codex 命令行界面之外执行。只有推理库在离线阶段使用这些调用，即每个训练任务两次，用于成功判断和记忆项提取。核心集选择在任何方法开始之前，还会为每个候选任务发出一次辅助大语言模型调用。该成本在匹配核心集上由各方法共享，并在本附录末尾报告一次。嵌入调用。推理库和行列式点过程核心集选择均调用

BAAI/bge-large-en-v1.5 (肖等人, 2023) 进行指纹或查询嵌入。我们

表 7: 在 SWE-Bench Pro ($k = 10$ 个训练任务, $M_{\text{test}} = 100$ 个留出任务) 上优化阶段按角色划分的智能体调用。ROLLOUT 是每个核心集任务 G 次并行求解, AFTER 是每个核心集任务每个候选一次求解, RANK 是每个核心集任务每个候选一次成对排序, TEST 是每个留出任务一次求解。计数直接从持久化轨迹目录中读取, 大语言模型报告辅助聊天补全调用。

Method	rollout	diagnose	optimize	after	rank	test	total	llm
Vanilla Codex	0	0	0	0	0	100	100	0
ReasoningBank (Ouyang et al., 2025)	10	0	0	0	0	100	110	20
Dynamic Cheatsheet (Suzgun et al., 2025)	30	0	10	10	10	100	160	0
Sleep-time Compute (Lin et al., 2025)	30	0	30	30	30	100	220	0
RHO	30	10	3	30	30	100	203	0

在本地执行编码器, 因此这些调用不会访问远程端点, 也不与大语言模型总数汇总。

G.2 各方法分解

表 7 将 SWE-Bench Pro ($k = 10$ 个训练任务, $M_{\text{test}} =$ 个留出任务) 上的优化阶段按角色分解为智能体调用。终端基准 2 仅在 $M_{\text{test}} = 59$, 上有所不同, 而 GAIA-2 与 SWE-Bench Pro 一致。该表是对每种方法如何花费其离线预算的字面核算。推理库对每个训练任务发出一次求解, 且从不提出候选测试框架, 因此离线阶段在留出评估之上仅花费 $k = 10$ 次额外的 Codex 调用, 其 $2k$ 次辅助大语言模型调用是成功判断器和记忆项提取器。动态备忘单和睡眠时间计算均对每个训练任务展开 $G = 3$ 次以暴露方差, 然后运行优化器智能体。动态备忘单对所有 k 个任务调用单个策展器 ($N=1$ 个候选), 而睡眠时间计算对每个任务每次重启运行一次编辑遍历 (3 次重启 $\times$ k 个任务 $=$ 30 次优化调用)。然后两者均通过与 RHO 共享的求解后和成对排序步骤提交每个唯一候选测试框架。RHO 的足迹与睡眠时间计算的不同之处仅在于将 $30 - 3 = 27$ 次每任务优化调用替换为 10 次诊断调用加上最佳 N 提议器中的 3 次候选级优化调用, 而求解后和排序阶段在 $N = 3$ 个候选上完全相同。

G.3 墙钟时间

表 8 报告了每个 (方法, 数据集) 单元格的两个互补的墙钟时间测量。 Σ_{codex} 是运行中每次代理调用的每次调用墙钟时间之和, 它是在一个代理进程上串行运行相同工作负载的成本。端到端是从运行开始到完成所经过的时间, 在我们的基础设施下

真实设置下为 10 个并发智能体调用和 10 个并发评分器。两列之间的比率反映了工作负载成功并行化的程度, 其中仅轨迹方法达到 $5 - 10\times$ 的加速比, 而推理库 (Reasoning-Bank) 达到 $3.5 - 4.5\times$, 因为其离线阶段按构造是顺序的 (每个训练任务的求解必须先提交到记忆, 然后才能检索下一个任务)。元测试平台 (Meta-Harness) 是一种验证反馈方法, 而非仅轨迹方法, 因此我们将其预算单独报告在 § G.4 中, 此处不列出。有两个观察值得明确说明。首先, 每次调用的成本在不同数据集之间差异超过 $2\times$ (GAIA-2 平均每次调用约 150 秒, 终端基准 2 (Terminal-Bench 2) 约 300 秒, 软件工程基准专业版 (SWE-Bench Pro) 约 400 秒), 因此智能体调用计数仅在单个数据集内有意义, 不能仅通过汇总调用次数来跨数据集比较成本。其次, 在匹配的核心集和 $N = 3$ 下, RHO 在每个数据集上与睡眠时间计算 (Sleep-time Compute) 相差在 $\pm 4\%$ 以内, 比动态备忘单 (Dynamic Cheatsheet) (使用 $N = 1$) 更昂贵, 并且比推理库 (ReasoningBank) (不执行群体展开、不提出候选、不进行成对秩排序) 昂贵得多。

G.4 匹配预算元集成

元哈内斯 (Meta-Harness) 的计算足迹在质量上更大, 因为验证评分会反馈给提议者。在其上游默认设置 ($I = 20$ 次外层迭代, 每次迭代 $C = 3$ 个候选, 每个任务 $T = 2$ 次求解尝试)

$$I(1 + CM_{\text{search}}T) + M_{\text{search}}T \approx 1,210$$

下, 仅离线阶段在 SWE-Bench Pro 上就会消耗智能体调用次数——几乎是 RHO 优化阶段计数 103 的十二倍。我们在表 2 中报告的匹配预算配置将其减少到

$$M_{\text{search}}T + I(1 + CM_{\text{search}}T) = I = 1, C = 3, T = 1$$

, 相当于 $10 + 31 = 41$ 次优化阶段调用; 这是其保留集通过率 (0.62 (在 SWE-Bench Pro 上) 被一并报告的配置

表 8: 按数据集划分的单次运行墙钟成本。 Σ_{codex} 是所有智能体调用中每次调用墙钟时间的总和（即一个 Codex 子进程上的成本），端到端时间是在 10 路智能体并发和 10 路评分器并发下从运行开始到完成的经过时间。单位：小时。

Method	SWE-Bench Pro		Terminal-Bench 2		GAIA-2	
	Σ_{codex}	end-to-end	Σ_{codex}	end-to-end	Σ_{codex}	end-to-end
Vanilla Codex	9.4	1.1	4.3	0.6	4.2	0.6
ReasoningBank (Ouyang et al., 2025)	17.5	4.6	8.0	2.4	7.2	2.0
Dynamic Cheatsheet (Suzgun et al., 2025)	17.9	2.5	10.8	1.9	6.5	1.2
Sleep-time Compute (Lin et al., 2025)	22.2	3.2	14.5	2.2	7.0	1.2
RHO	23.1	3.2	15.5	2.5	9.2	1.4

主文中的 RHO。同一表格中报告的 10 轮配置保持 $C = 3$, $T = 1$ 不变, 但将 I 提升至 10, 从而得到 $10 + 10 \cdot 31 = 320$ 次优化阶段调用 ($3.1 \times$ RHO 的优化阶段计数为 103)。这些计数不包括共享的 $M_{\text{test}} = 100$ 留出评估, 无论优化器如何, 每种方法都需支付一次该评估成本。由于 Meta-Harness 是一种验证反馈方法而非仅轨迹方法, 因此未纳入表 8 的墙钟时间比较。

G.5 核心集选择成本（共享）

核心集选择在任何方法的离线阶段开始前运行一次。对于 SWE-Bench Pro, 这消耗 100 次辅助 LLM 调用（每个候选任务一次难度判断）加上对相同 100 个指纹的一次批量嵌入调用, 之后选定的 $k = 10$ 任务 ID 被持久化并被所有基线逐字复用。此成本每个数据集支付一次, 不归属于任何单个方法。

H 优化后的测试框架工件

本附录逐字复现 RHO 在每个基准上产生的最高分测试框架的完整内容，即图 3 中总结的测试框架。每个测试框架是一个包含 Markdown 指令和技能文件以及可执行工具脚本的目录；每个文件均完整展示，无删节。文件按基准分组，并标注其在测试框架目录内的路径及其角色（指令、技能或工具）。

H.1 SWE-Bench Pro

Listing 6 SWE-Bench Pro harness: README.md (instructions)

```
# SWE-bench Repair Harness
```

在编辑前和最终确定任务前使用此测试框架。目标是使最终补丁准确回答提示，而不仅仅是编译通过。

```
## Default Workflow
```

1. 将提示视为契约。编写一份简短的需求清单，涵盖：确切的公共名称、文件路径、载荷结构、错误/状态码、默认值、支持的后端/平台以及明确指出的边缘情况。
2. 当任务听起来像已知修复时，寻找现有参考：本地 git 历史、标签、上游提交、测试、固定装置或附近的实现模式。优先采用经过验证的行为，而非猜测。
3. 在编辑之前，端到端地追踪真实路径。将存储的数据追溯到用户可见的工件，将配置值追溯到其运行时表示，将包装器追溯到实际执行的实现。
4. 进行满足契约的最小生产源代码更改。将生成的文件、缓存、临时冒烟测试和本地环境输出排除在提交的差异之外。
5. 验证可执行行为。语法检查是不够的：至少运行一个聚焦的冒烟测试，调用每个新的公共路由/函数/方法，并覆盖提示中的每个显式失败/边缘条件。
6. 在最终答案之前重新打开需求清单。检查字面 API 名称、响应形状、默认值、清理行为以及提示中提到的每个后端或平台。

```
## Tools
```

- ‘harness/bin/repair-verify’发现常见的非 PATH 工具链，运行廉价的语言检查，报告生成的工件，并接受额外的任务特定命令。
- ‘harness/checklists/contract-verification.md’ is the detailed checklist for requirement-led smoke tests and final review.

Typical use:

```
```bash
../harness/bin/repair-verify repo ../harness/bin/repair-verify repo -- go test ./lib/events ./lib/service
../harness/bin/repair-verify repo --clean-pycache -- python -m compileall -q lib```
```

如果项目依赖不可用，仍然使用本地存根为更改的层编写一个小的直接冒烟测试。冒烟测试应执行隐藏测试可能断言的契约形状。

#### 清单 7 SWE-Bench Pro 测试框架：检查清单/合同验证.md（技能）

```
Contract Verification Checklist
```

将此作为每次修复的最终关卡。先前运行中的高严重性遗漏源于未执行确切的提示合同。

```
Requirement Ledger
```

- 从提示中复制确切的接口名称：类型/函数/方法名称、结构体字段、路由路径、配置键、默认值、错误字符串和负载形状。
- 将每个需求标记为以下之一：已实现、有意保持不变并附原因，或受阻并附观察到的证据。
- 在最终确定前重新检查字面措辞。隐藏测试通常针对确切名称进行编译，例如‘StreamEmitter’、‘getRaw’、‘BackoffDuration’或‘durationSeconds’。

## ## Behavioral Smoke Tests

- 直接测试每个新的公共入口点，而不仅仅是语法或导入检查。对于 Web/API 工作，调用控制器或路由层并断言响应体形状和错误转换。
- 覆盖提示中的负面路径：缺失记录、权限被拒绝、队列已满、发射器已关闭、资源已删除、平台不受支持、截止日期已过、后端失败以及边界位置。
- 对于状态机，断言在开始、中间、结束、停止、暂停/播放以及无效输入边界处的转换。
- 对于数据导入/解析任务，将完整的真实夹具输出与预期输出进行比较。只有在至少一个真实回归夹具完全通过后，合成示例才有用。
- 对于文件系统/数据清理任务，追踪生成或转换的工件。如果存储的记录已规范化，也要测试可见/生成的文件名。
- 对于可配置行为，测试项目使用的运行时配置值形状，包括适用时的字符串值，如“0”和“1”。
- 对于多后端特性，对每个命名的后端/路径以及每个显式选择器值进行冒烟测试。不要只验证最容易运行的后端。

## ## Language/Repo Recipes

- Go：在声明 Go 不可用之前，搜索‘/tmp/go/bin/go’和‘/tmp/go/bin/gofmt’。对修改过的文件运行 gofmt，运行聚焦的包测试，并在提示提到某个平台时编译跨平台构建标签路径。
- Python：run ‘python -m py\_compile’ or ‘compileall’ on touched files, then a direct smoke for changed behavior. Remove ‘\_\_pycache\_\_’, ‘.pyc’,  
↪ ‘.pytest\_cache’,  
and test environment artifacts before finalizing.
- JavaScript/TypeScript：语法/类型检查无法捕获缺失的运行时导入或错误的响应形状。当完整依赖缺失时，为更改的应用、控制器或模型方法添加一个带有桩的小型 Node 冒烟测试。
- 缺失文件任务：在本地/上游历史中搜索目标文件及其测试。在适配文件后运行窄范围的包测试或上游测试。

## ## Final Diff Hygiene

- 运行‘git diff --check’。
- 运行‘git status --short --untracked-files=all’并检查未跟踪的文件。
- 移除生成的缓存、本地冒烟文件、下载的夹具、虚拟环境、node 缓存、覆盖率输出和临时日志，除非提示明确要求添加它们。
- 若任何常规测试无法运行，请说明所执行的确切命令及观察到的  原因，然后以可执行的最接近的直接冒烟测试进行补偿。

## 清单 8 SWE-Bench Pro 测试框架：bin/repair-verify (工具)

```
#!/usr/bin/env bash
set -euo pipefail
```

```
usage() { cat <<'EOF' 用法：修复验证 [仓库] [--清理字节码缓存] [-- 额外命令...]
```

在 SWE-Bench 仓库内部运行低成本、任务无关的验证辅助工具：

- 发现常见的非 PATH 环境下的 Go/Python/Node 工具
- 当仓库为 git 工作树时运行 git diff --check

- 当存在明显的源文件时运行低成本的语法/包检查
- 报告通常应移除的生成工件
- 可选地运行一个额外的任务特定命令

示例：harness/bin/repair-verify 仓库 harness/bin/repair-verify 仓库 -- 运行测试 ./lib/events ./lib/service  
harness/bin/repair-verify 仓库 --清理字节码缓存 -- 编译所有 Python 文件 -q lib 文件结束 }

```
repo="repo"
clean_pycache=0

while [[$# -gt 0]]; do
 case "$1" in
 --help|-h)
 usage
 exit 0
 ;;
 --clean-pycache)
 clean_pycache=1
 shift
 ;;
 --)
 shift
 break
 ;;
 -*)
 echo "unknown option: $1" >&2
 usage >&2
 exit 2
 ;;
 *)
 repo="$1"
 shift
 ;;
 esac
done
```

如果 [[ ! -d "\$repo" ]]; 那么 echo "仓库目录未找到: \$repo" >&2 退出 2 结束

```
cd "$repo"
```

```
find_tool() { 局部 name="$1" 移位 如果 command -v "$name"
>/dev/null 2>&1; 那么 command -v "$name" 返回 0 结束 对于候选
在 "$@"; 做 如果 [[-x "$candidate"]]; 那么 printf "%s\n" "$candidate"
返回 0 结束 完成 返回 1 }
```

```
go_bin="$(find_tool go /tmp/go/bin/go || true)"
gofmt_bin="$(find_tool gofmt /tmp/go/bin/gofmt || true)"
python_bin="$(find_tool python3 /usr/bin/python3 /usr/local/bin/python3 ||
↪ find_tool python /usr/bin/python || true)"
node_bin="$(find_tool node /usr/bin/node /usr/local/bin/node || true)"
```



回显 "==" 已发现的工具 == " 回显 "go: \${go\_bin:-not 已找到}" 回显 "gofmt: \${gofmt\_bin:-not 已找到}" 回显 "python: \${python\_bin:-not 已找到}" 回显 "node: \${node\_bin:-not 已找到}"

如果 `git rev-parse --is-inside-work-tree >/dev/null 2>&1`; 那么 `echo echo "==" git 差异检查`  
`==" git diff --check` 结束

```
if [[-n "$gofmt_bin"]] && find . -path ./git -prune -o -name '*.go' -print \
 ↪ -quit | grep -q .; then
 echo
 echo "==" gofmt check == "
 unformatted="$(find . -path ./git -prune -o -name '*.go' \
 ↪ -print)"
 if [[-n "$unformatted"]]; then
 echo "$unformatted"
 echo "gofmt check failed: run gofmt on the files above" >&2
 exit 1
 fi
fi

if [[-n "$go_bin" && -f go.mod]]; then
 echo
 echo "==" go package list == "
 "$go_bin" list ./... >/dev/null
fi

if [[-n "$python_bin"]] && find . -path ./git -prune -o -name '*.py' -print \
 ↪ -quit | grep -q .; then
 echo echo "==" python compile touched files == " if git rev-parse --is-inside-work-tree >/dev/null 2>&1; then mapfile -t
 py_files <<(git diff --name-only --diff-filter=ACMR -- '*.py') if [[${#py_files[@]} -gt 0]]; then "$python_bin" -m py_compile
 "${py_files[@]}" else echo "no changed Python files" fi else echo "not a git worktree; skipping touched-file compile" fi fi

if [[-n "$node_bin"]] && git rev-parse --is-inside-work-tree >/dev/null 2>&1; \
 ↪ then
 mapfile -t js_files <<(git diff --name-only --diff-filter=ACMR -- '*.js' \
 ↪ '*.cjs' '*.mjs')
 if [[${#js_files[@]} -gt 0]]; then
 echo
 echo "==" node syntax touched js files == "
 for file in "${js_files[@]}; do
 "$node_bin" --check "$file"
 done
 fi
fi

if [[$# -gt 0]]; then
 echo
 echo "==" extra command == "
 "$@"
fi

if [["$clean_pycache" -eq 1]]; then
 echo
 echo "==" cleaning python caches == "
```

```

find . \(-type d -name __pycache__ -o -type d -name .pytest_cache \) -prune
↪ -exec rm -rf {} +
find . -type f \(-name '*.pyc' -o -name '*.pyo' \) -delete
fi

echo
echo "== generated artifact scan =="
find . \
 -path ./git -prune -o \
 \(-type d \(-name __pycache__ -o -name .pytest_cache -o -name .mypy_cache
 ↪ -o -name .tox -o -name node_modules -o -name .nyc_output -o -name coverage
 ↪ \) \
 -o -type f \(-name '*.pyc' -o -name '*.pyo' -o -name '*.profrw' -o
 ↪ '*.test' \) \) \
 -print | sed -n 's#^\. /##;1,200p' -name

if git rev-parse --is-inside-work-tree >/dev/null 2>&1; then
 echo
 echo "== worktree status =="
 git status --short --untracked-files=all
fi

echo
echo "Reminder: run a direct smoke for the prompt contract: public API shape,
↪ 负路径、命名后端/平台以及边界情况。”

```

## H.2 Terminal-Bench 2

### Listing 9 Terminal-Bench 2 harness: README.md (instructions)

# Terminal-Bench Harness Playbook

Use this harness before solving a task and again before the final answer. The

↪ 目标是依据任务本身的要求解决该问题，将所要求的 ↪ 工件保留在容器中，并避免在仅 ↪ 满足表面症状的路径上浪费时间。

## First Pass

1. 从提示中提取契约：

- 最终工件及其确切路径，通常位于'/app'下
- 评分器可能使用的确切命令或调用方式
- 允许和禁止的文件：将'test/'和'tests/'视为不同名称
- 任务是黑盒/仅 API、逆向工程还是包管理

↪ repair, system setup, or stochastic implementation

2. Use the host-to-container script pattern from the prompt for nontrivial

↪ commands. It avoids quoting failures and leaves a useful command trail in  
↪ '/host-ws'.

3. Inspect only sources that are consistent with the task. "Readable" is not

↪ the same as "methodologically allowed."

4. Start with a tiny reproducible probe, then scale to the full input. Save

↪ 中间脚本、日志和验证命令保留在主机工作区中 ↪ 在有用时。

## Integrity Boundaries

For tasks framed as black-box recovery, model extraction, API querying, or "by

↪ 进行查询：

- 不得读取实现内部细节以恢复隐藏参数、常量，

↪ labels, or answers.

↪ explicitly asks for source inspection. 口，除非提示

- 使用查询衍生的证据验证恢复：留出探针、↪ 递归/支持诊断、往返检查或输出一致性。即使隐藏的真实变量被暴露，也 ↪ 不得与其进行比较。

For tasks asking for an emulator, interpreter, compiler, sampler, server, or

↪ 独立重新实现：

- 优先采用通用实现路径，而非任务特定的状态注入或  
     ↳ host-side artifact synthesis.  
     - 若必须使用宿主钩子，保持其最小化并记录其为何保持  
     ↳ the requested abstraction.
- 验证最终工件是否通过预期路径生成。例如：一个  仿真程序应通过仿真系统调用自行创建其输出， 而非通过宿主机对内部内存的重建。

## ## Final Validation Gate

在结束前，运行一次面向评分器的验证：

- 运行提示中给出的确切命令，使用相同的标志和路径。
- 比较所有重要的可观察输出：文件、标准输出、标准错误、退出码。  
     ↳ status, sizes, and checksums when applicable.  
     - 验证工件位于将被评分的容器文件系统路径中。  
     ↳ not only in `/host-ws`.  
     - 确认未使用禁止路径和评分资产。
- 当可见示例为时，至少添加一项隐藏表面冒烟检查。 收窄：导入更多模块，练习第二种应用程序编程接口形式，测试 加入/发布/离开而非仅设置，或事后检查几何约束  
     ↳ serialization.

## ## Package And Container Repair

Use this for Python/R/Cython/library compatibility tasks.

- 保持提示所要求的依赖版本稳定。注意‘pip  
     ↳ install` or build tools silently upgrading NumPy or other pinned packages.  
     - 从源码树外部运行所请求的代码片段，使导入来自  
     ↳ the installed package.  
     - 断言编译扩展从编译工件加载，而非回退到‘.py’  
     ↳ files:

“‘sh python - <<’PY’ 导入 inspect 从包中导入  
模块 打印 (inspect.getfile(module)) PY”

- 在精确代码片段之后添加广泛的非评分器检查：
  - 导入具有代表性的公共模块，排除禁止的测试/评分器路径  
     ↳ compatibility is a risk - 对包代码执行‘compileall’，在语法检查时排除测试目录
  - 直接为已编译辅助函数发起冒烟调用
- Reuse `tools/python\_package\_smoke.py` for broad import, compiled-extension,  
     ↳ and compile checks when it fits the package shape.
- 优先采用显式的、针对特定文件的兼容性补丁，而非宽泛的正则表达式  重写，除非重写之后会进行扫描以检查意外更改。

## ## Reverse Engineering And Binary Matching

- 首先收集行为信息：退出状态、标准输出、标准错误、生成的文件、哈希值，  
     ↳ file sizes, and compression limits.
- 使用确切的提示命令重新构建。优化标志可能会改变  
     ↳ floating-point or layout-sensitive output.
- 对每个可观察工件进行逐字节比较。对于生成的图像或二进制文件，  
     ↳ compare both hashes and targeted structure fields.
- 如果存在源大小限制，请检查来自  
     ↳ prompt.

## ## Large File Editing And Vim Macros

- 在之前，根据头部、尾部、随机样本和行数推断转换  
     ↳ editing the real file.  
     - 根据允许的命令列表和计数宏验证脚本形状  
     ↳ keystrokes.

- 对于简单的正则变换，优先采用文件级替换；逐行  $\leftrightarrow$  宏在百万行规模下可能慢得多。

- 使用 Vim 替换字符串时，在
  - $\hookrightarrow$  tiny copy first.
  - 若 `/usr/bin/time` 不可用，使用可移植的计时回退方案，如 `'date +%s'`
- $\hookrightarrow$  unavailable.
- 最终检查：在保存的副本或真实输入上运行无头 Vim，然后执行 `'cmp'`
- $\hookrightarrow$  against the expected file.

## ## Geometry And Segmentation Outputs

对于掩码转换或图像几何任务：

- 当提示要求真实权重时，使用所需的模型系列和真实权重
  - $\hookrightarrow$  segmentation. A randomly initialized checkpoint only validates plumbing.
- 保持命令行界面（CLI）通用性：不硬编码演示路径；支持提示中的
  - $\hookrightarrow$  positional/flag style when ambiguous.
  - 在写入 CSV/JSON 输出后，验证序列化工件，而不仅仅是
    - $\hookrightarrow$  内存中的掩码：
- 行数和非凡列保持不变
- 所有掩码均为折线，当格式要求闭合时闭合
- 无矩形类回退多边形
- 每个对象恰好一个连通分量
- 轮廓导出后无栅格化重叠
- Reuse `'tools/validate_mask_csv.py'` for CSV polygon masks when the column
  - $\hookrightarrow$  names match the prompt.
- 若中央处理器（CPU）推理速度较慢，可对库中相互独立的模型调用进行批处理
  - $\hookrightarrow$  supports it.

## ## Metacircular Or Language Tasks

- 将可见的“测试/”程序视为经认可的示例，除非提示
  - $\hookrightarrow$  forbids that exact path. Do not touch `'tests/'`, `'test_outputs'`, `grader`
  - $\hookrightarrow$  logs, or solution files.
  - 将原生解释器输出与新的求值器/解释器进行对比
  - $\hookrightarrow$  every visible program.
  - 将提示中的嵌套/自托管示例作为最终冒烟检查纳入其中。
- 除非提示中另有说明，否则避免对评估器自身文件名进行特殊处理  $\leftrightarrow$  明确允许这一点；它削弱了自我解释确实存在的证据。

## ## System Service Tasks

- 优先使用常规服务命令行界面，而非低层次的数据库/应用程序编程接口对象创建，
  - $\hookrightarrow$  because defaults often populate domains, styles, policies, queues, and
  - $\hookrightarrow$  pipelines.
  - 启动守护进程前验证所有权和运行时目录。
  - $\hookrightarrow$  direct behavior. - 不要只信任单一的状态命令；还要检查进程、队列、日志和
- 从提示构建完整的行为冒烟测试。对于邮件列表，这意味着  $\leftrightarrow$  加入确认、向订阅者投递、离开  $\leftrightarrow$  确认、队列排空以及最终策略值。

## ## Scientific Or Stochastic Implementations

- 确认运行时存在，如果缺失则安装所需的最小运行时。
  - $\hookrightarrow$  does not.
- The public `'test()'` function should print clear `'NAME: PASS'` or `'NAME: FAIL'`
  - $\hookrightarrow$  lines plus relevant statistics.
  - 测试公共采样器和任何复杂的辅助模块。
- 包含确定性的无效输入检查以及至少一个拒绝案例。
  - $\hookrightarrow$  for violated assumptions, such as non-log-concavity.
- 对于随机输出，比较分布统计量并设置容差， $\leftrightarrow$  持久化所需的样本文件。

## ## Constraint/Solver Recovery Tasks

- 对于小型密码或精确的有限状态系统，当比手工推导完整攻击更简单时，将精确操作编码为约束。  $\leftrightarrow$

- 在可行的情况下，约束所有可用的已知示例，而不仅仅是获得一个候选所需的最少示例。 ↩
- 针对每个已知对验证恢复的参数，然后使用提供的加密/解密或往返路径验证最终交付物。 ↩

#### Listing 10 Terminal-Bench 2 harness: tools/python\_package\_smoke.py (tool)

```
#!/usr/bin/env python3
"""Run installed-package smoke checks without touching test/grader paths."""

from __future__ import annotations

import argparse
import compileall
import importlib
import inspect
import pkgutil
from pathlib import Path

FORBIDDEN_PARTS = {
 "test",
 "tests",
 "test_outputs",
 "solution",
 "logs",
 "verifier",
 "__pycache__",
}

def forbidden(path: Path) -> bool:
 return any(part in FORBIDDEN_PARTS for part in path.parts)

def iter_modules(package_name: str, limit: int | None) -> list[str]:
 package = importlib.import_module(package_name)
 names = [package_name]
 package_path = getattr(package, "__path__", None)
 if package_path is not None:
 for module in pkgutil.walk_packages(package_path,
↩ prefix=f"{package_name}."):
 lower_parts = set(module.name.lower().split("."))
 if lower_parts & FORBIDDEN_PARTS:
 continue
 names.append(module.name)
 if limit is not None and len(names) >= limit:
 break
 return names

def main() -> int:
 parser = argparse.ArgumentParser()
 parser.add_argument("package")
 parser.add_argument("--compiled-module", action="append", default=[])
 parser.add_argument("--import-limit", type=int, default=200)
 parser.add_argument("--compileall", action="store_true")
 args = parser.parse_args()

 失败: 字符串列表 = [] 已导入 = 0 对于名称在 iter_modules(args.package, args.import_limit): 尝试:
 importlib.import_module(name) 已导入 += 1 除了异常作为 exc: # noqa: BLE001 - 冒烟工具应报告所有

 ↩ failures
```



```
failures.append(f'导入失败 {name}: {type(exc).__name__}: {exc}')
```

对于 args.compiled\_module: 中的每个名称, 尝试: 模块 = importlib.import\_module(name) 路径 = inspect.getfile(模块) 除了异常作为 exc: # noqa: BLE001 失败项.append(f'编译模块导入失败 {名称}: {type(exc).\_\_name\_\_}: {exc}') 继续 如果没有任何 (路径.endswith(后缀) 对于后缀在 (".so", ".pyd", ".dll", ".dylib")): 失败项.append(f'{名称} 未从编译扩展加载: {路径}')

如果 args.compileall: 包 = importlib.import\_module(args.package) 根目录 = [Path(路径) 对于路径在 getattr(包, "\_\_path\_\_", []) 对于根目录中的每个根目录: 对于 py\_file 在 root.rglob("\*.py") 中: 如果 forbidden(py\_file): 继续 成功 = compileall.compile\_file(str(py\_file), quiet=1) 如果不成功: 失败项.append(f'编译失败 {py\_file}')

如果失败项: print(f'PYTHON\_PACKAGE\_SMOKE: 失败 imported={imported}') 对于失败项[:80] 中的每个失败: 打印 (失败) 如果 len(失败项) > 80: 打印 (f'... {len(失败项) - 80} 更多失败') 返回 1

```
print(f'PYTHON_PACKAGE_SMOKE: PASS imported={imported}')
return 0
```

```
if __name__ == "__main__":
 raise SystemExit(main())
```

#### Listing 11 Terminal-Bench 2 harness: tools/validate\_mask\_csv.py (tool)

```
#!/usr/bin/env python3
"""Validate polygon mask CSV outputs for segmentation/refinement tasks.
```

检查器在结构上是刻意的: 它不评判分割质量, 但会捕获序列化后常见的评分表面失败。"""

```
from __future__ import annotations

import argparse
import ast
from pathlib import Path

import numpy as np
import pandas as pd

def parse_coords(value: object) -> list[float]:
 if isinstance(value, (list, tuple, np.ndarray)):
 return [float(x) for x in value]
 text = str(value).strip()
 if not text:
 return []
```

尝试: 解析 = ast.literal\_eval(text) 除了 (ValueError, SyntaxError): 文本 = 文本.strip("[]") 如果非文本: 返回 []  
返回 [float(部分) 对于部分在 文本.replace(";", ";").split(",") 如果  $\rightarrow$  部分.strip() 如果非 isinstance(解析, (list, tuple)):

```
 return []
 return [float(x) for x in parsed]
```

```
def is_rectangle_like(xs: list[float], ys: list[float]) -> bool:
 points = list(zip(xs, ys))
 if len(points) < 4:
 return False
 unique = list(dict.fromkeys(points))
 if len(unique) == 5 and unique[0] == unique[-1]:
 unique = unique[:-1]
 if len(unique) != 4:
 return False
 return len({x for x, _ in unique}) == 2 and len({y for _, y in unique}) == 2
```

```
def rasterize_with_numpy(points: np.ndarray, height: int, width: int) ->
 $\rightarrow$ np.ndarray:
```

```
 mask = np.zeros((height, width), dtype=np.uint8)
 min_x = max(0, int(np.floor(points[:, 0].min())))
 max_x = min(width - 1, int(np.ceil(points[:, 0].max())))
 min_y = max(0, int(np.floor(points[:, 1].min())))
 max_y = min(height - 1, int(np.ceil(points[:, 1].max())))
 if min_x > max_x or min_y > max_y:
 return mask
```

$x\_centers = np.arange(min\_x, max\_x + 1, dtype=float) + 0.5$  对于  $y$  在  $range(min\_y, max\_y + 1)$ :  $y\_center = y + 0.5$   
内部 =  $np.zeros\_like(x\_centers, dtype=bool)$   $j = len(\text{点}) - 1$  对于  $i$  在  $range(len(\text{点}))$  中:  $x_i, y_i = \text{点}[i]$   $x_j, y_j = \text{点}[j]$   
交叉 =  $(y_i > y\_center) \neq (y_j > y\_center)$  如果交叉:  $x\_intersect = (x_j - x_i) * (y\_center - y_i) / (y_j - y_i) + x_i$   
内部  $\wedge = x\_centers < x\_intersect$

```
 j = i
 mask[y, min_x : max_x + 1] = inside.astype(np.uint8)
 return mask
```

```
def rasterize(xs: list[float], ys: list[float], height: int, width: int) ->
 $\rightarrow$ np.ndarray:
```

```
 pts = np.array([[round(x), round(y)] for x, y in zip(xs, ys)],
 $\rightarrow$ dtype=np.int32)
 pts[:, 0] = np.clip(pts[:, 0], 0, width - 1)
 pts[:, 1] = np.clip(pts[:, 1], 0, height - 1)
 if len(pts) < 3:
 return np.zeros((height, width), dtype=np.uint8)
 try:
 import cv2
```

掩码 =  $np.zeros((\text{高度}, \text{宽度}), dtype=np.uint8)$   $cv2.fillPoly(\text{掩码}, [\text{点集}], 1)$  返回 掩码 除了  
ImportError: 返回 rasterize\_with\_numpy(pts.astype(float), 高度, 宽度)

```
def connected_components(mask: np.ndarray) -> int:
 try:
 import cv2
```

```
 count, _ = cv2.connectedComponents(mask.astype(np.uint8),
 ↪ 连通性=8) 返回整数 (计数 - 1) 除导入错误外: 已见 = 零数组 (掩码.形状, dtype=bool) 活跃 = 掩码.转
 为布尔型 组件 = 0 高度, 宽度 = 掩码.形状 对于 y, x 在 数值.按条件索引 (活跃) 中: 若 已见 [y, x]: 继续
 组件 += 1 栈 = [(整数 (y), 整数 (x))] 已见 [y, x] = 真 当 栈 非空时: 当前 y, 当前 x = 栈.弹出() 对于 新 y 在
 范围 (最大值 (0, 当前 y - 1), 最小值 (高度, 当前 y + 2)) 中: 对于 新 x 在 范围 (最大值 (0, 当前 x - 1), 最小值
 (宽度, 当前 x + 2)) 中: 若 活跃 [新 y, 新 x] 且 非 已见 [新 y, 新 x]: 已见 [新 y, 新 x] = 真 栈.追加 ((新 y, 新
 x)) 返回 组件
```

```
def infer_canvas(df: pd.DataFrame, margin: int) -> tuple[int, int]:
 xmax = int(np.ceil(pd.to_numeric(df["xmax"]).max())) + margin
 ymax = int(np.ceil(pd.to_numeric(df["ymax"]).max())) + margin
 return max(1, ymax), max(1, xmax)
```

```
def main() -> int:
 parser = argparse.ArgumentParser()
 parser.add_argument("--input-csv", help="Optional original CSV for
 ↪ row/column preservation checks")
 parser.add_argument("--output-csv", required=True)
 parser.add_argument("--width", type=int)
 parser.add_argument("--height", type=int)
 parser.add_argument("--margin", type=int, default=3)
 args = parser.parse_args()
```

```
 output_path = Path(args.output_csv) 数据框 = pd.read_csv(output_path) 必需 = {"xmin", "xmax",
 "ymin", "ymax", "coords_x", "coords_y"} 缺失 = 必需 - 集合 (数据框.列) 失败列表: 字符串列表 = [] 若
 缺失: 失败列表.追加 (f"缺失列: {排序 (缺失)}") print("\n".join(failures)) 返回 1
```

```
 若 args.input_csv: 原始 = pd.read_csv(args.input_csv) 若 长度 (原始) != 长度 (数据框): 失败列表.追加
 (f"行数变化: input={len(original)}")
```

```
 ↪ output={len(df)}")
 lost = [col for col in original.columns if col not in df.columns]
 if lost:
 failures.append(f"output lost input columns: {lost}")

 height, width = (
 (args.height, args.width)
 if args.height and args.width
 else infer_canvas(df, args.margin)
)
```

```

占用 = np.zeros((高度, 宽度), dtype=np.uint16) 对于索引, 行在 df.iterrows(): 横坐标 = parse_coords(row["坐标
x"]) 纵坐标 = parse_coords(row["坐标y"]) 如果长度 (横坐标) != 长度 (纵坐标): failures.append(f"行 {索引}:
coords_x/coords_y 长度不匹配") 继续 如果长度 (横坐标) < 4: failures.append(f"行 {索引}: 少于 4 个多边形
点") 继续 if (xs[0], ys[0]) != (xs[-1], ys[-1]): failures.append(f"行 {索引}: 多边形未闭合
") 如果 is_rectangle_like(xs, 纵坐标): failures.append(f"行 {索引}: 类似矩形的多边形") 掩码 = 栅格化 (横坐
标, 纵坐标, 高度, 宽度) 组件 = connected_components(mask) 如果组件 != 1: failures.append(f"行 {索引}: 连
通 components={components}") 占用 += 掩码.astype(np.uint16)

```

```

overlap_pixels = int((occupied > 1).sum())
if overlap_pixels:
 failures.append(f"raster overlap pixels={overlap_pixels}")

```

若失败列表: print("MASK\_CSV\_VALIDATION: 失败") 对于失败 在 失败列表 [:50]  
 中: 打印 (失败) 若 长度 (失败列表) > 50: 打印 (f"... {长度 (失败列表) - 50} 更多失败  
 ") 返回 1

```

print(f"MASK_CSV_VALIDATION: PASS rows={len(df)} canvas={width}x{height}")
return 0

```

```

if __name__ == "__main__":
 raise SystemExit(main())

```

### H.3 GAIA-2

#### Listing 12 GAIA-2 harness: README.md (instructions)

```
GAIA-2 Harness Notes
```

Use this as the first checklist for future GAIA-2 ARE tasks. The recurring

↔ 先前运行中的失败在写入前是弱证据, 模糊的 ↔ 引用、精确的时间/引用文本漂移、遗漏的强制副作用、↔ 以及嘈杂的工具发现。

```
First Minute
```

1. Read the user request through AgentUserInterface, not the Codex prompt or  
↔ 仅'轮询':

```

```bash
python task/tools/are.py call AgentUserInterface get_all_messages --json '{}'
```

```

2. Read `task/tools/catalog.json` for app and function names. Avoid dumping the  
↔ 完整的“are.py list”输出, 除非目录缺失或损坏。3. 在解析相对日期、时间窗口、↔ “今天”、“本月”或截止  
日期计算之前, 获取模拟的当前时间:

```

```bash
python task/tools/are.py call SystemApp get_current_time --json '{}'
```

```

4. 对于每个所需函数，在调用之前检查其精确模式：

```
“bash python task/tools/arc.py schema <AppName> <function_name> “
```

5. Do not read `task/.gaia2/scenario.json`. Avoid broad `.gaia2\_state` or

↔ 边车日志检查：除非正常应用程序编程接口无法暴露 ↔ 所需证据，否则使用应用程序编程接口。

## Request Decomposition

在写入之前，制作一个简短的私人清单：

- 强制写入：每次发送、创建、删除、订单、移动、预订或最终  
↔ user reply that must happen.
- 条件写入：确切条件、所需证据以及如果  
↔ condition is not met.
- 模糊引用：任何可能匹配多个电子邮件、文件的短语，  
↔ people, events, products, or conversations.
- 精确细节：引号字符串、标点符号、数量、服务类型，  
↔ addresses, dates, times, folders, and recipients.
- 验证：每次写入后进行一次回读调用，然后执行`python ↔ 任务/工具/arc.py`在可能触发环境回复的写入后进行轮询。

Do not let a conditional branch swallow a later mandatory instruction. A

↔ 诸如“然后发邮件给 X 告知出租车已预订”这样的句子仍然是强制性的 ↔ 除非用户明确将其设为条件。

## Evidence Rules Before Acting

- 切勿仅依据文件名发送、移动、删除、回复、订购或预订，  
↔ loose search hits, or an inferred heuristic.
  - 对于文件任务，通过内容确认目标。如果区域文件读取返回  
↔ 内容为空或延迟加载，请使用目录/模式查找另一个 ↔ 内容/搜索/读取操作并继续搜索。如果内容仍然 ↔ 不可用，则将选择视为未解决，而不是根据 ↔ 文件名进行猜测。
  - 对于电子邮件任务，搜索结果可能匹配正文文本以及主题。如果  
↔ the user asks for a subject/title, read candidate metadata and manually  
↔ filter by subject.
    - 对于聊天/消息，将搜索结果标识解析回参与者名称  
↔ before acting.
    - 对于产品变体，验证可用性、确切的变体属性，  
↔ quantity, and price/order state after checkout.
- 对于日历持续时间/计数比较，在相关范围内分页所有事件，并在一个小脚本中一次性计算总数，而不是目测部分视图。↔ 范围并一次性计算总数，而不是目测 ↔ 部分视图。

## Ambiguity Policy

当用户引用有多个合理目标时：

1. Enumerate the candidates with stable identifiers and distinguishing metadata.
2. If the user told you to clarify on issues, ask through AgentUserInterface  
↔ 并且仅在任务给出截止时间或环境允许的情况下等待  
↔ convention requires.
3. For irreversible external actions such as sending email, deleting  
↔ 文件/事件、订购产品或预订乘车时，在未澄清后不要发明 ↔ 后备方案。完成无歧义的部分，通过代理用户界面（AgentUserInterface）向用户报告 ↔ 未解决的阻碍，并且不要 ↔ 执行有歧义的写入。
4. 仅当用户指定了后备方案或应用数据给出了

↔ 单一无歧义的、有文档记录的规则时，才使用后备方案。

## Timing And Waiting

Treat timing constraints as first-class requirements.

- 根据用户消息时间戳或当前模拟时间计算绝对截止时间
  - ↪ `time before doing other work.`
  - 在备注中保留精确的目标时间，例如“在 2024-10-15 回复”
    - ↪ `07:04:00; add to cart at 07:04:30".`
- 预先解析所有目标 ID、负载、产品变体和引用的字符串 ↪ 在等待截止时间之前。等待结束后，直接执行截止时间写入 ↪，中间不进行额外的搜索或模式发现。
- 如果多个写入共享同一个精确时间戳，则使用最快的直接调用路径
  - ↪ `available. Avoid per-item searching between writes; prepare the call`
  - ↪ `arguments first.`
- “等待”和通知调用可能会提前返回或推进模拟时间
  - ↪ `chunks. Do not run background or wall-clock loops that keep advancing time`
  - ↪ 在你操作期间。
    - 优先选择有界等待到下一个绝对截止时间，然后立即执行
      - ↪ `the deadline action.`
- 监控窗口时，按时间戳过滤/读取消息并忽略
  - ↪ `messages outside the requested window, even if they appear during an extra`
  - ↪ `check.`
- 对于邮件后回复，通过时间戳、父/线程标识、发件人以及 ↪ 需要时进行完整收件箱分页。

Exact quoted text matters. Preserve punctuation inside quotes, including  
 ↪ 尾随逗号，并通过读回验证创建的标题/正文。

## ## Pagination And Aggregation

Many list APIs are paginated or have view limits. Page until exhaustion, not  
 ↪ 直到第一个有用命中：

- 推进偏移/页面令牌时保持相同过滤器。
  - 当返回空页面或页面条目数少于
    - ↪ `requested limit.`
- 当任务要求“最多”时，在最终答案前记录计数证据，
  - ↪ `"how many", "latest", or "which app has more".`
- Use `'harness/are_helper.py page ...'` when a function uses simple
  - ↪ `'limit'/'offset'` arguments.

## ## User-Facing Protocol

- 法典最终答案仅为运行摘要，不计入通用人工智能评估-2
  - ↪ `user reply.`
  - 通过智能体用户界面发送进度/完成/阻塞消息
    - ↪ `'send_message_to_user'.`
- 在最终智能体用户界面消息之前，确认每项强制写入要么已完成 ↪ 并验证，要么因条件为假而有意跳过，要么
  - ↪ 因明确未解决的歧义而受阻。

## ## Helper Script

`'harness/are_helper.py'` wraps common ARE calls and avoids shell quoting/JQ  
 ↪ mistakes.

Examples:

```
"bash python harness/are_helper.py catalog --app Emails python harness/are_helper.py schema Emails
search_emails python harness/are_helper.py call AgentUserInterface send_message_to_user
```

```
↪ --json '{"content":"Done"}'
python harness/are_helper.py page Contacts list_contacts --limit 100 --json '{}'
```

Use the helper for convenience, but still inspect the schema for exact argument  
 ↪ names.



### Listing 13 GAIA-2 harness: are\_helper.py (tool)

```
#!/usr/bin/env python3
"""Small ARE command helper for GAIA-2 tasks.
```

该脚本有意保持通用性：它通过外壳调用任务/工具/are.py，在 Python 内部保持 JSON 引号，并为具有标准分页参数的应用程序编程接口提供简单的偏移/极限分页器。"""

```
from __future__ import annotations
```

```
import argparse
import json
import subprocess
import sys
from pathlib import Path
from typing import Any
```

```
COMMON_ITEM_KEYS = ("
项目", "结果", "数据", "记录",
"消息", "邮件", "事件", "联系
人", "对话", "产品", "订单", "
行程", "文件", "条目",)
```

```
def parse_json_arg(raw: str) -> dict[str, Any]:
 try:
 value = json.loads(raw)
 except json.JSONDecodeError as exc:
 raise SystemExit(f"--json is not valid JSON: {exc}") from exc
 if not isinstance(value, dict):
 raise SystemExit("--json must decode to an object")
 return value
```

```
def are_py(task_root: str) -> Path:
 path = Path(task_root) / "tools" / "are.py"
 if not path.exists():
 raise SystemExit(f"ARE tool not found: {path}")
 return path
```

```
def run_are(task_root: str, args: list[str]) -> Any:
 cmd = [sys.executable, str(are_py(task_root)), *args]
 proc = subprocess.run(cmd, text=True, capture_output=True)
 if proc.returncode != 0:
 sys.stderr.write(proc.stderr)
 sys.stderr.write(proc.stdout)
 raise SystemExit(proc.returncode)
```

输出 = 进程标准输出去除首尾空白，若  
输出为空则返回空值，否则尝试以 JSON  
格式解析输出，若解析失败则抛出 JSON  
解码错误：

```

 return out

def print_json(value: Any) -> None:
 print(json.dumps(value, indent=2, ensure_ascii=False, sort_keys=True))

def command_catalog(args: argparse.Namespace) -> None:
 path = Path(args.task_root) / "tools" / "catalog.json"
 if not path.exists():
 raise SystemExit(f"Catalog not found: {path}")
 catalog = json.loads(path.read_text())
 if args.app is None:
 print_json(catalog)
 return

 needle = args.app.lower()

 def keep(value: Any) -> Any:
 if isinstance(value, dict):
 kept: dict[str, Any] = {}
 for key, child in value.items():
 child_kept = keep(child)
 if needle in str(key).lower() or child_kept not in (None, {}),
 ↪ []):
 kept[key] = child if needle in str(key).lower() else
 ↪ child_kept
 return kept
 if isinstance(value, list):
 kept_list = [item for item in (keep(item) for item in value) if
 ↪ item not in (None, {}, [])]
 return kept_list
 if needle in str(value).lower():
 return value
 return None

 print_json(keep(catalog))

def command_schema(args: argparse.Namespace) -> None:
 print_json(run_are(args.task_root, ["schema", args.app, args.function]))

def command_call(args: argparse.Namespace) -> None:
 payload = parse_json_arg(args.json)
 print_json(
 run_are(
 args.task_root,
 ["call", args.app, args.function, "--json", json.dumps(payload,
 ↪ ensure_ascii=False)],
)
)

def extract_items(result: Any, explicit_key: str | None) -> list[Any]:
 if explicit_key:
 if not isinstance(result, dict) or explicit_key not in result:
 raise SystemExit(f"Result does not contain items key
 ↪ {explicit_key!r}")
 items = result[explicit_key]
 elif isinstance(result, list):
 items = result
 elif isinstance(result, dict):
 items = None
 for key in COMMON_ITEM_KEYS:
 if isinstance(result.get(key), list):
 items = result[key]
 break
 if items is None:

```

list\_values = [如果 `isinstance` 是列表实例，则取结果值中的每个值] 如果 len(list\_values) == 1: 项目 = 列表值 [0] 否则: 引发系统退出 (“无法推断列表字段。请使用 --items-key 重新运行。” f“顶层键: {', '.join(result.keys())}")  
否则: 引发系统退出 (f“无法对类型为 {type(result).\_\_name\_\_} 的非列表结果进行分页”)

若项不是列表，则抛出系统退出异常，提示推断出的项值不是列表，并返回项。

```
def command_page(args: argparse.Namespace) -> None:
 base = parse_json_arg(args.json)
 offset = args.start
 all_items: list[Any] = []
 pages = 0
```

进入无限循环: 构造负载 = 字典 (基础参数) payload[args.limit\_key] = 参数限制 payload[args.offset\_key] = 偏移量, 调用结果 = run\_are( args.task\_root, ["call", 参数应用, 参数函数, "--json", 将负载转为 JSON 字符串 ( `ensure_ascii=False` 确保 ASCII 为假) ], 获取项 = extract\_items(result, args.items\_key) all\_items.extend(items) 页数 += 为 1, 若项为空或项数小于参数限制则跳出循环, 若 args.max\_pages 不为空且页数 >= args.max\_pages: 则跳出循环, 偏移量 += 增加参数限制。

```
print_json({"count": len(all_items), "pages": pages, "items": all_items})
```

```
def build_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--task-root", default="task", help="Task root
 containing tools/are.py")
 sub = parser.add_subparsers(dest="command", required=True)

 catalog = sub.add_parser("catalog", help="Print catalog.json, optionally
 filtered by app text")
 catalog.add_argument("--app")
 catalog.set_defaults(func=command_catalog)
```

```
模式 = sub.add_parser("schema", help="Print 一个函数模式") schema.add_argument("app")
schema.add_argument("function") schema.set_defaults(func=command_schema)
```

```
调用 = sub.add_parser("call", help="Call 一个带有 JSON 参数的 ARE 函数") call.add_argument("app")
call.add_argument("function") call.add_argument("--json", default="{}") call.set_defaults(func=command_call)
```

```
page = sub.add_parser("page", help="Collect all pages for offset/limit list
APIs")
```

```
page.add_argument("app") page.add_argument("function") page.add_argument("--json", default={}, help="Base
排除 限制 / 偏移量 的 JSON 参数 ") page.add_argument("--limit", type=int, default=100)
page.add_argument("--start", type=int, default=0) page.add_argument("--limit-key", default="limit")
page.add_argument("--offset-key", default="offset") page.add_argument("--items-key")
page.add_argument("--max-pages", type=int) page.set_defaults(func=command_page) 返回解析器
```

```
def main() -> None:
 parser = build_parser()
 args = parser.parse_args()
 args.func(args)

if __name__ == "__main__":
 main()
```